

SDR

Estampilles

Mutex de Lamport

Olivier Lemer

↓ **Exclusion Mutuelle ?**

Exclusion Mutuelle

Définition

Module permettant à un bloc d'opérations de n'être exécuté que par **un processus à la fois**.

Ce bloc d'opérations est appelé **Section Critique** (SC).

Exemple inspiré de la syntaxe Java

```
synchronized {
```

```
}
```

Section Critique

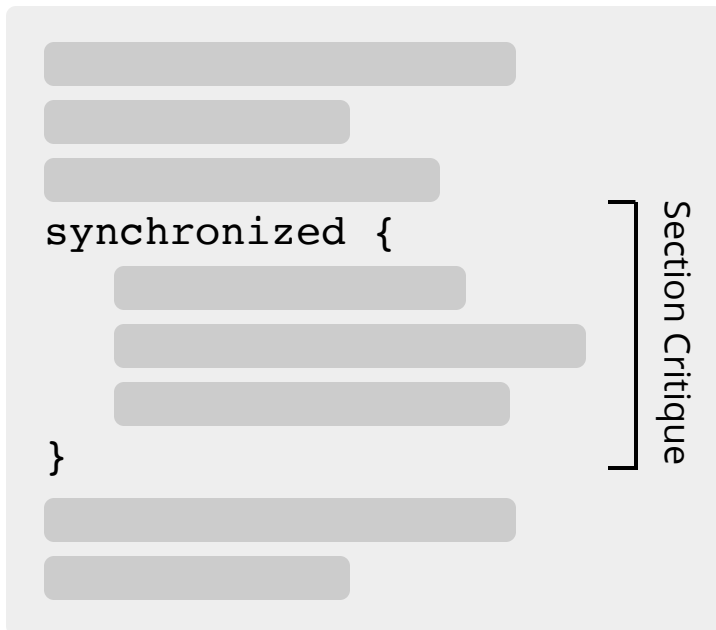
Exclusion Mutuelle

Définition

Module permettant à un bloc d'opérations de n'être exécuté que par **un processus à la fois**.

Ce bloc d'opérations est appelé **Section Critique** (SC).

Exemple inspiré de la syntaxe Java



Synchronisation nécessaire

- par locks (**shared memory**), ou
- par messages (**systèmes répartis**).

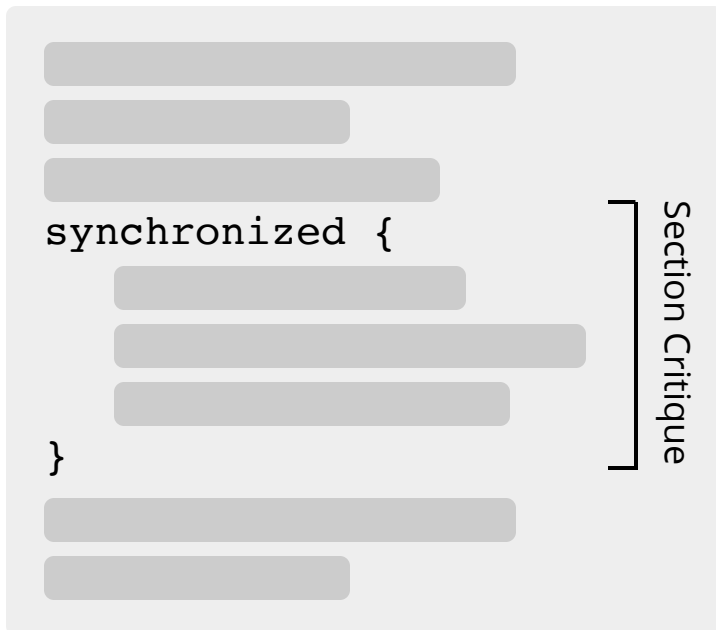
Exclusion Mutuelle

Définition

Module permettant à un bloc d'opérations de n'être exécuté que par **un processus à la fois**.

Ce bloc d'opérations est appelé **Section Critique** (SC).

Exemple inspiré de la syntaxe Java



Synchronisation nécessaire

- par locks (**shared memory**), ou
- par messages (**systèmes répartis**).

Exemple

Copie synchronisée d'un même compte en banque. Certaines opérations doivent être séquentielles.

Exclusion Mutuelle

Définition

Module permettant à un bloc d'opérations de n'être exécuté que par **un processus à la fois**.

Ce bloc d'opérations est appelé **Section Critique** (SC).

Exclusion Mutuelle

Définition

Module permettant à un bloc d'opérations de n'être exécuté que par **un processus à la fois**.

Ce bloc d'opérations est appelé **Section Critique** (SC).

Propriétés

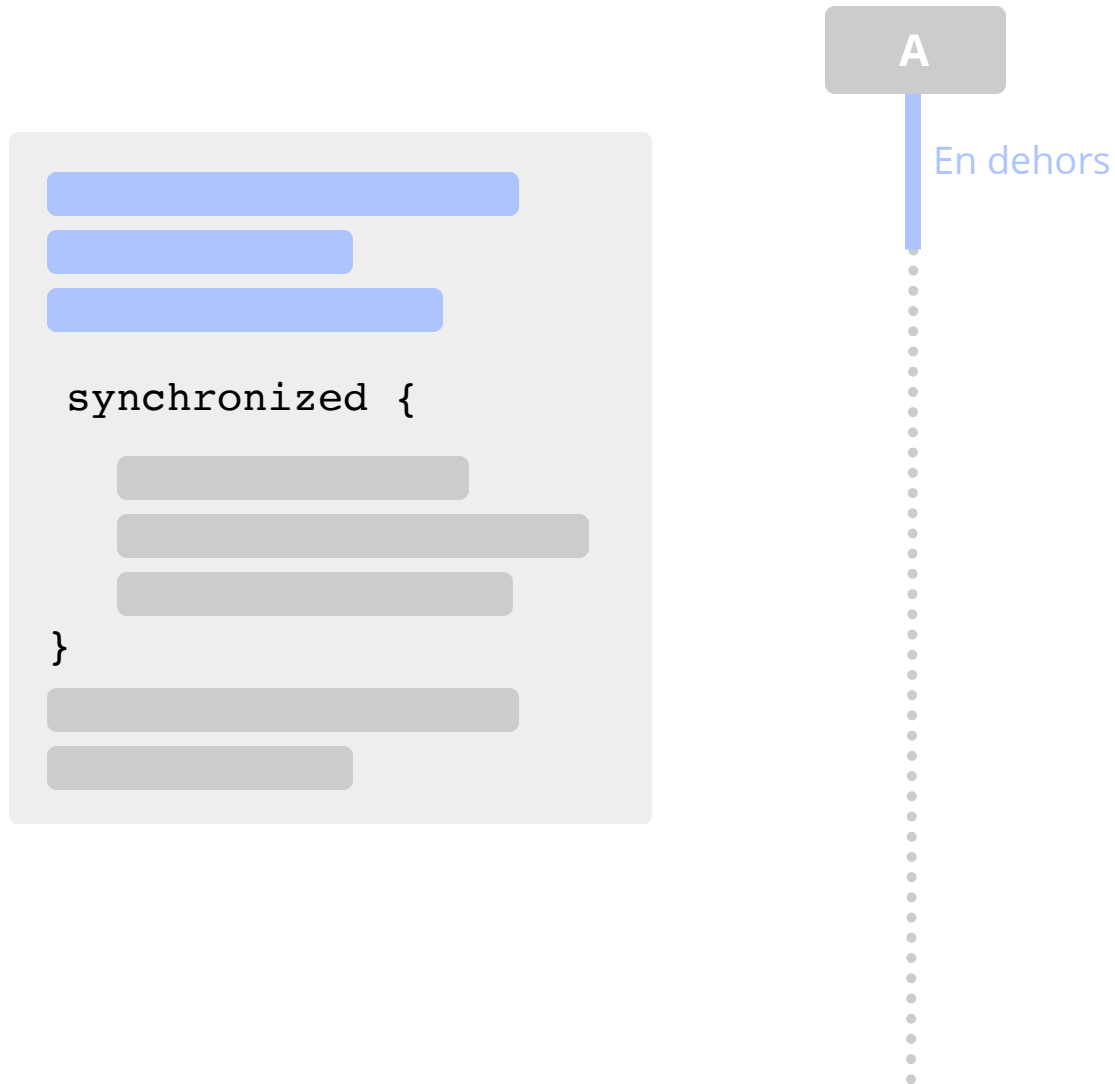
Correctness	Jamais plus d'un processus ne sera en SC à un instant
Progrès	† Toute demande d'entrée en SC sera autorisée un jour.

↓ **Comment ?**

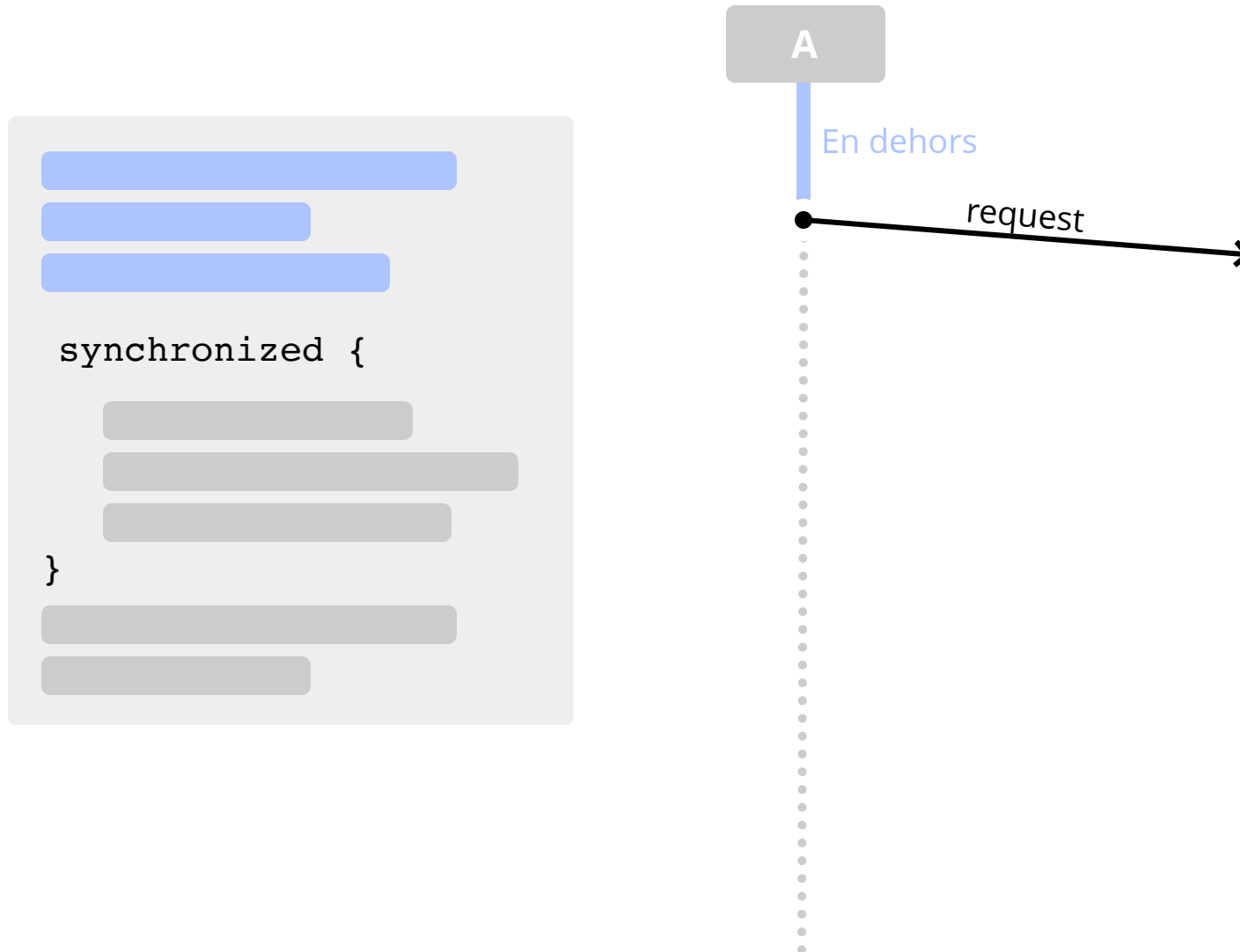
Synchronisation par messages



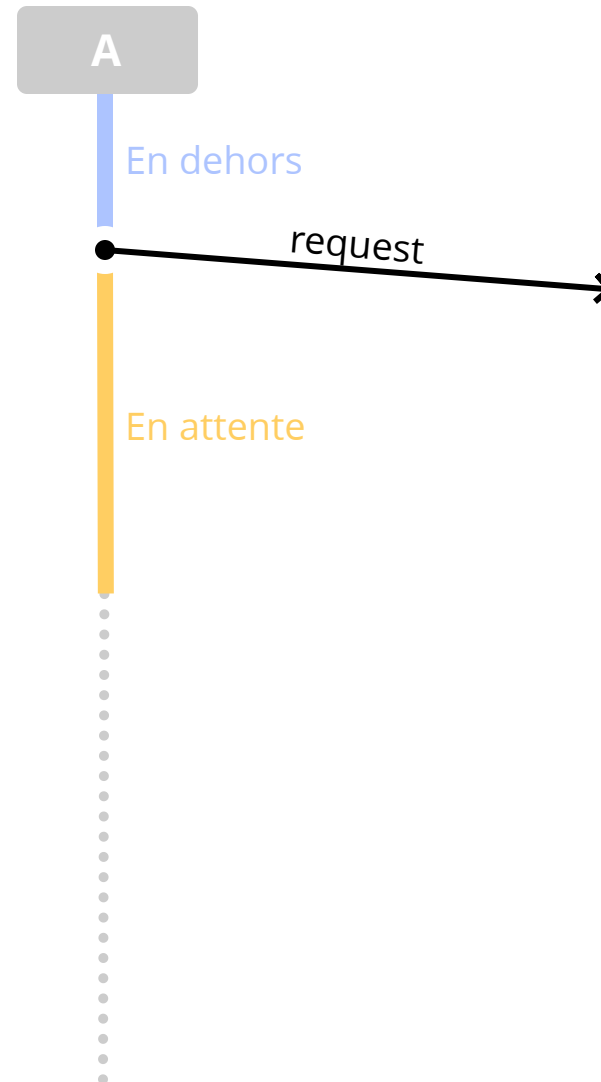
Synchronisation par messages



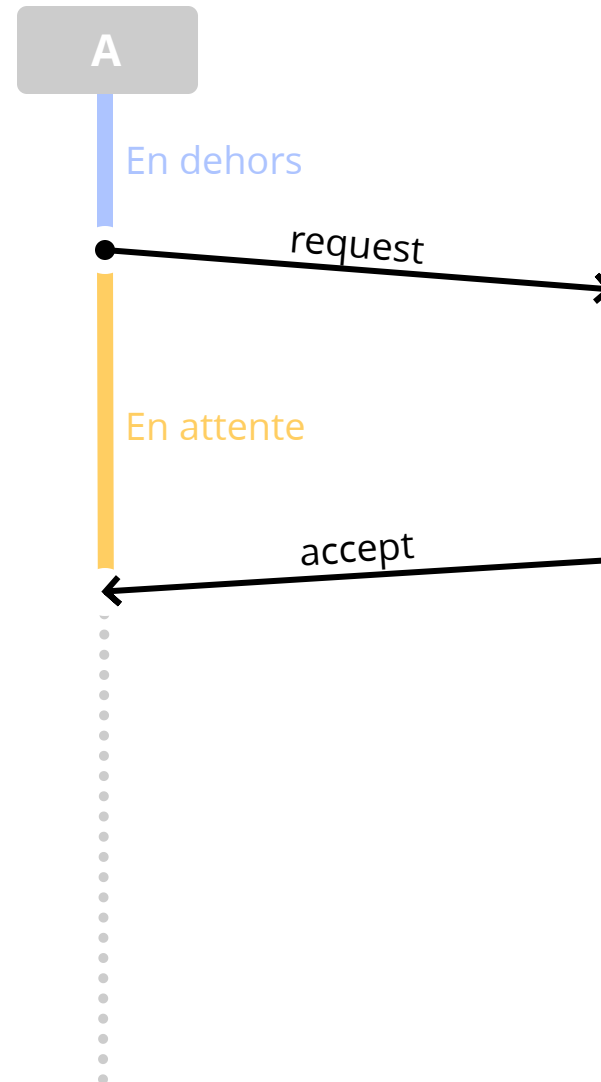
Synchronisation par messages



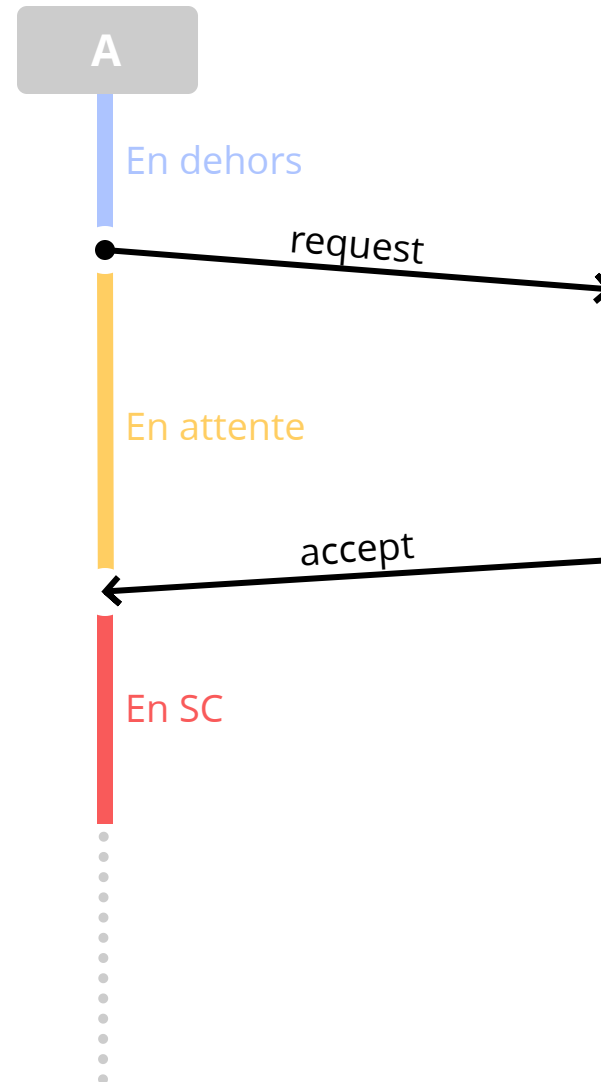
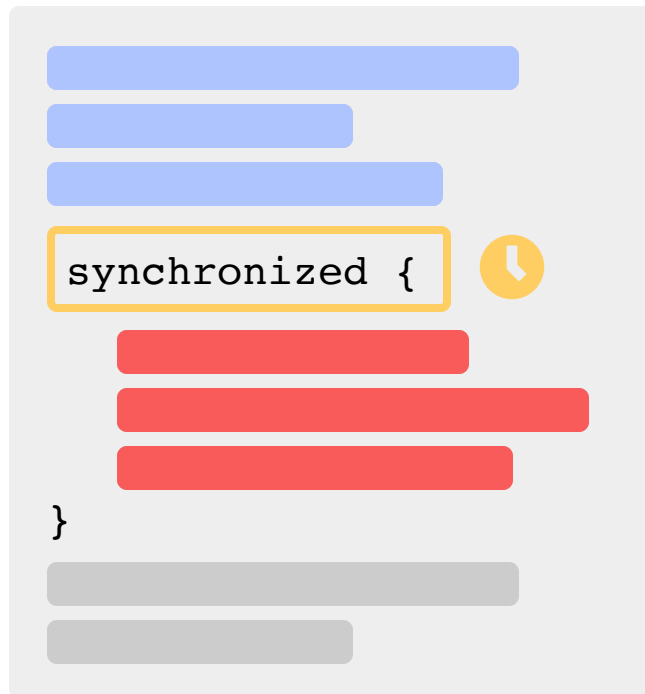
Synchronisation par messages



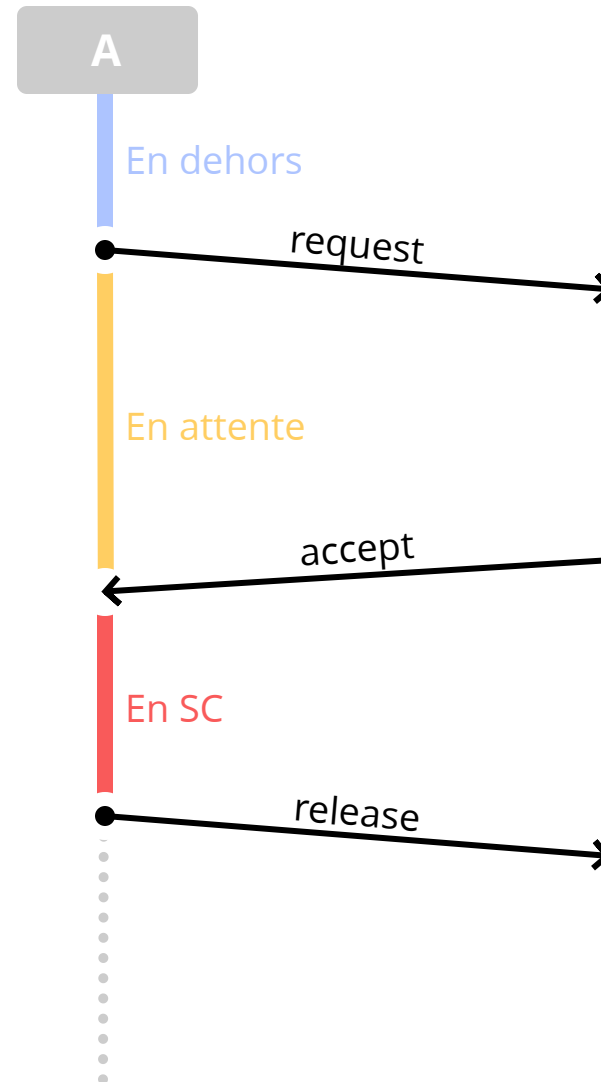
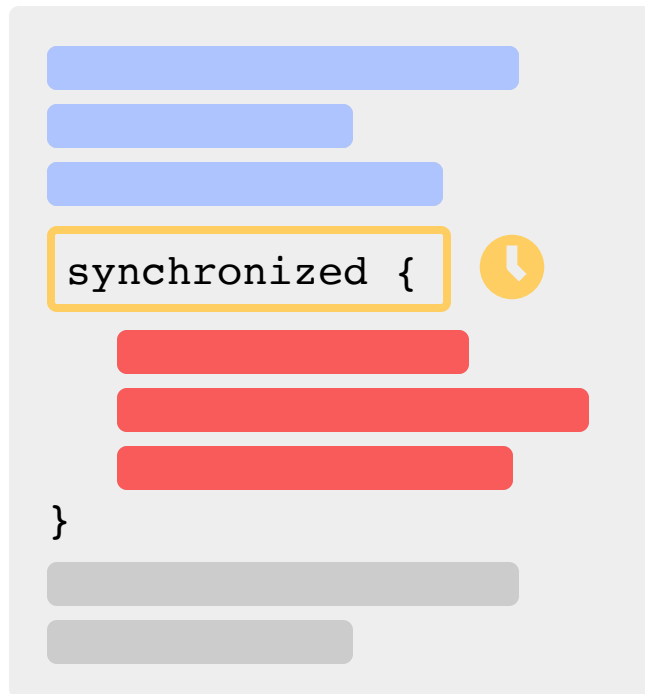
Synchronisation par messages



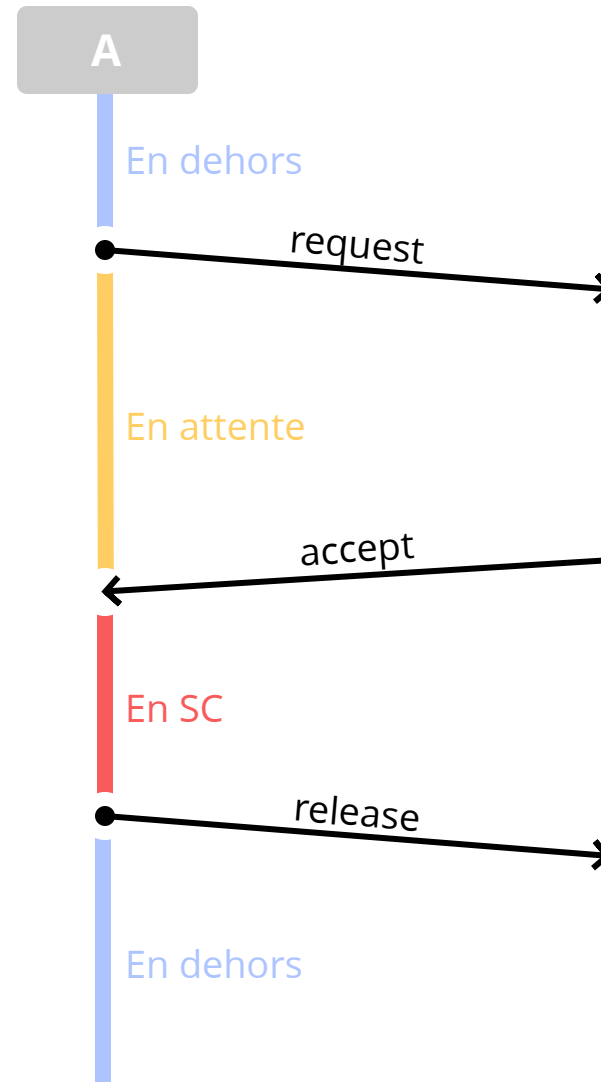
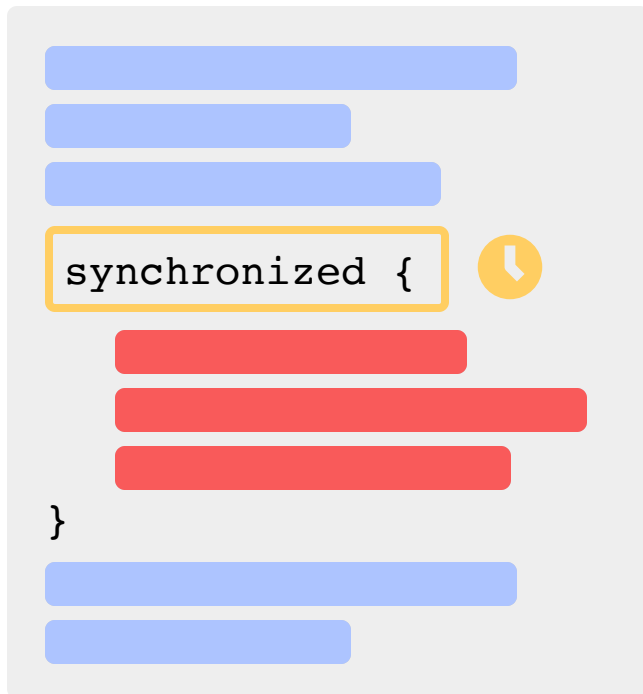
Synchronisation par messages



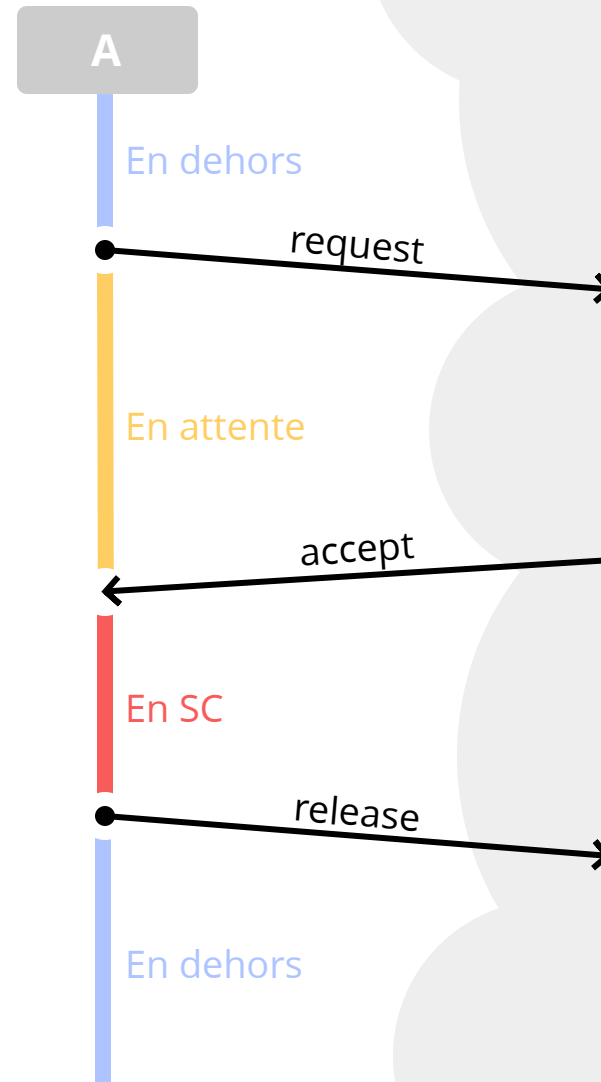
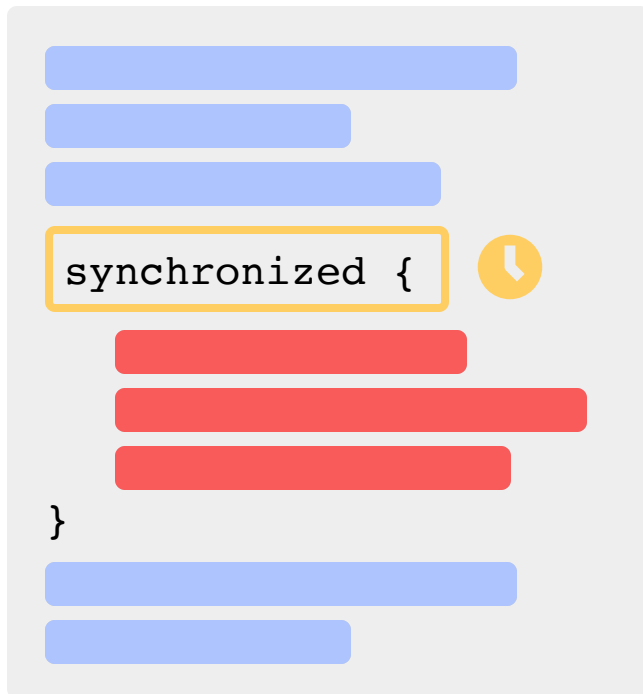
Synchronisation par messages



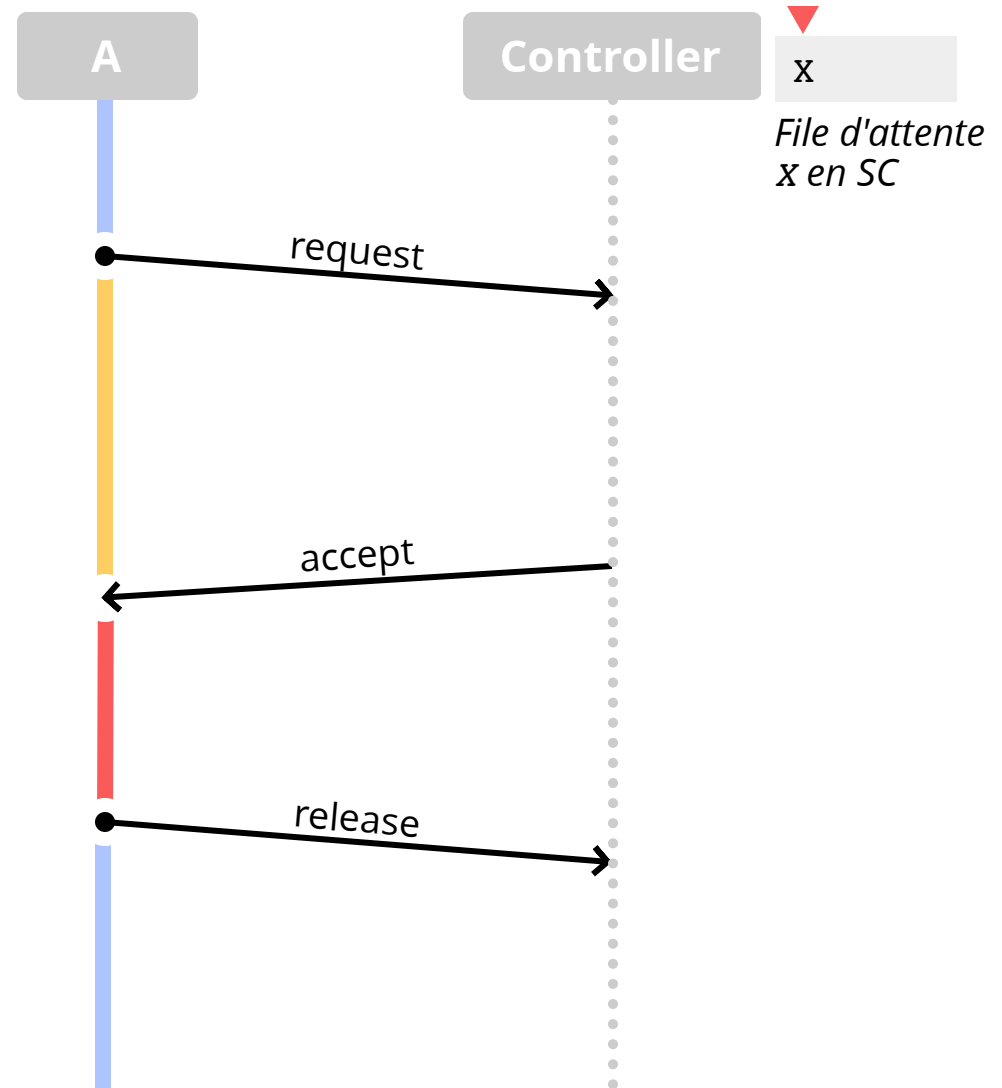
Synchronisation par messages



Synchronisation par messages

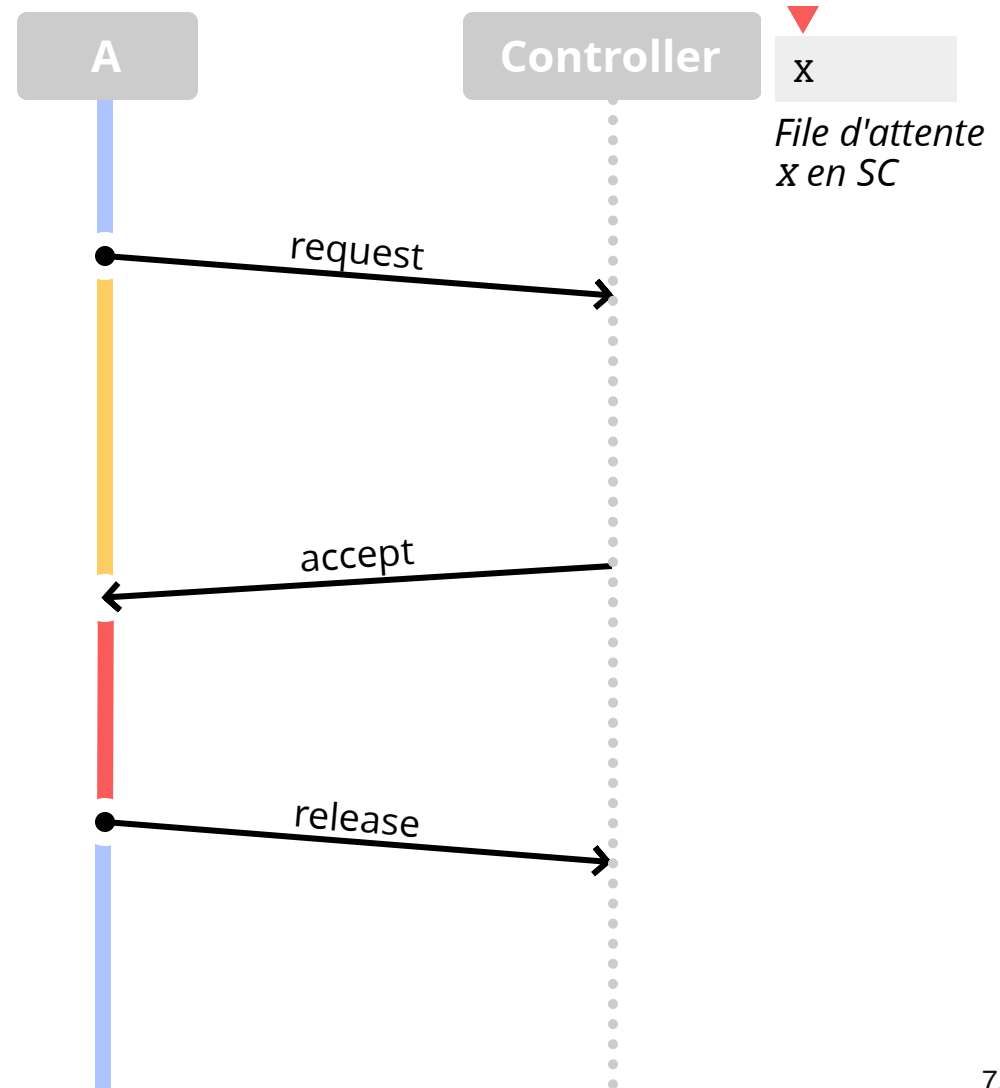


Solution centralisée



Solution centralisée

La tête de la file est toujours celle actuellement en SC.

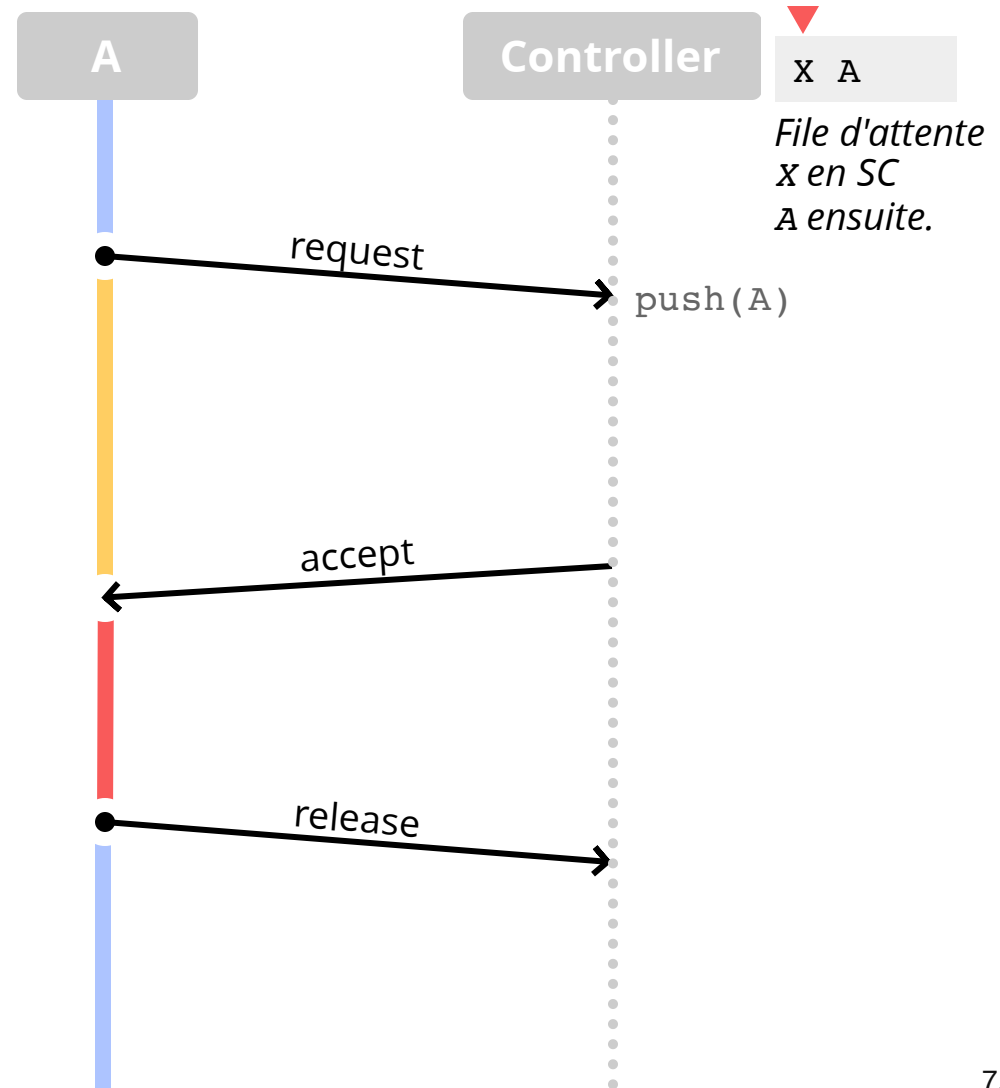


Solution centralisée

La tête de la file est toujours celle actuellement en SC.

On request

Push demandeur dans la file



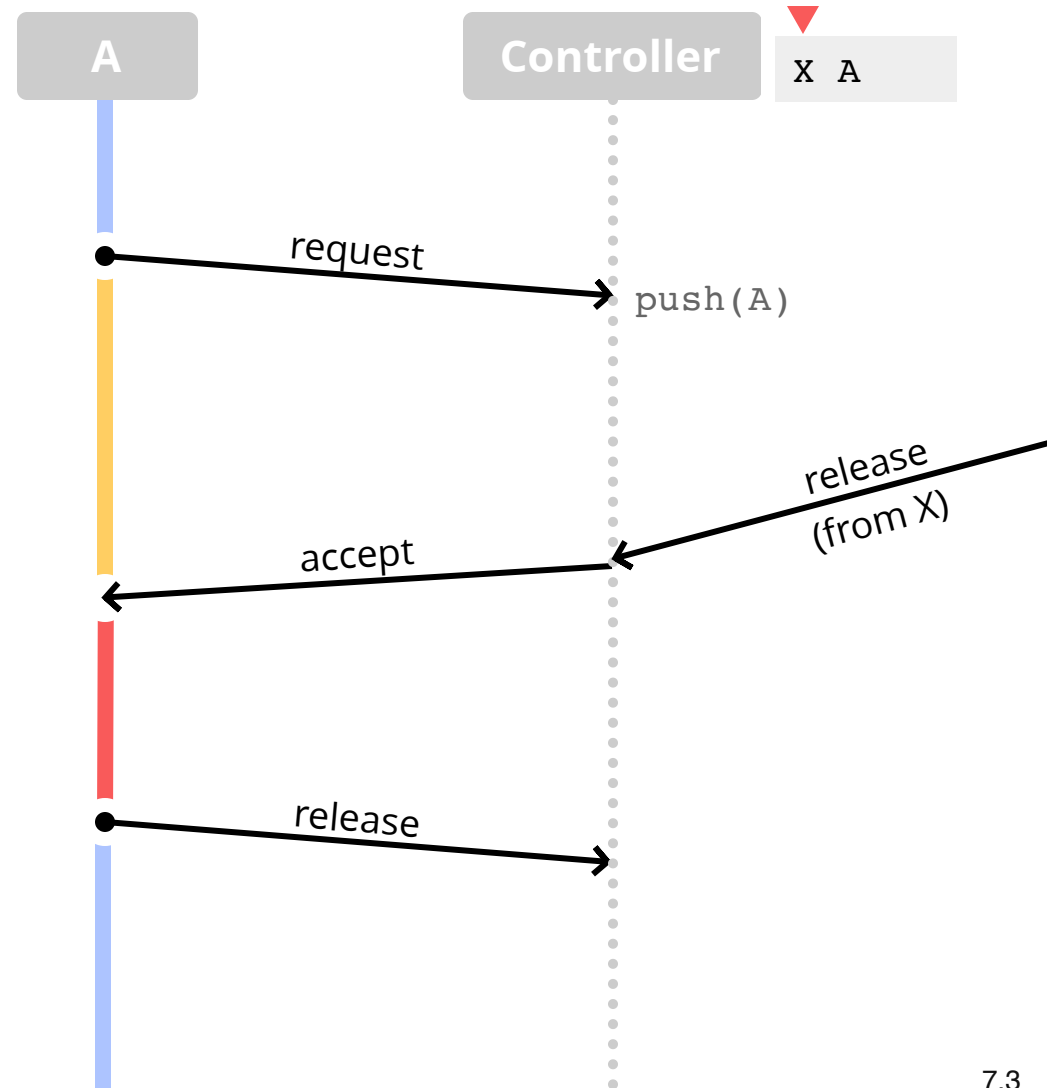
Solution centralisée

La tête de la file est toujours celle actuellement en SC.

On request

Push demandeur dans la file

On release



Solution centralisée

La tête de la file est toujours celle actuellement en SC.

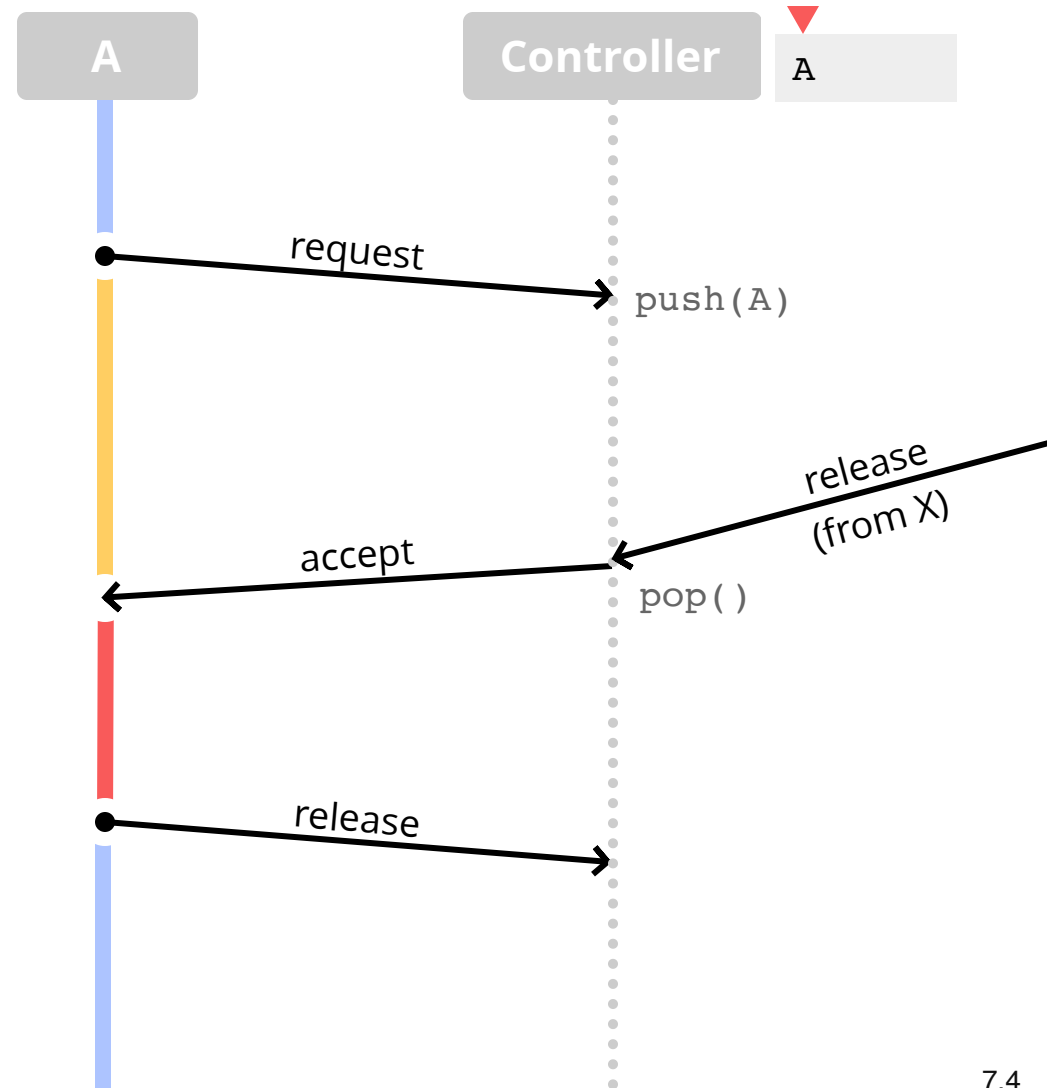
On request

Push demandeur dans la file

On release

Pop tête de file

Envoi de accept à nouvelle tête



Solution centralisée

La tête de la file est toujours celle actuellement en SC.

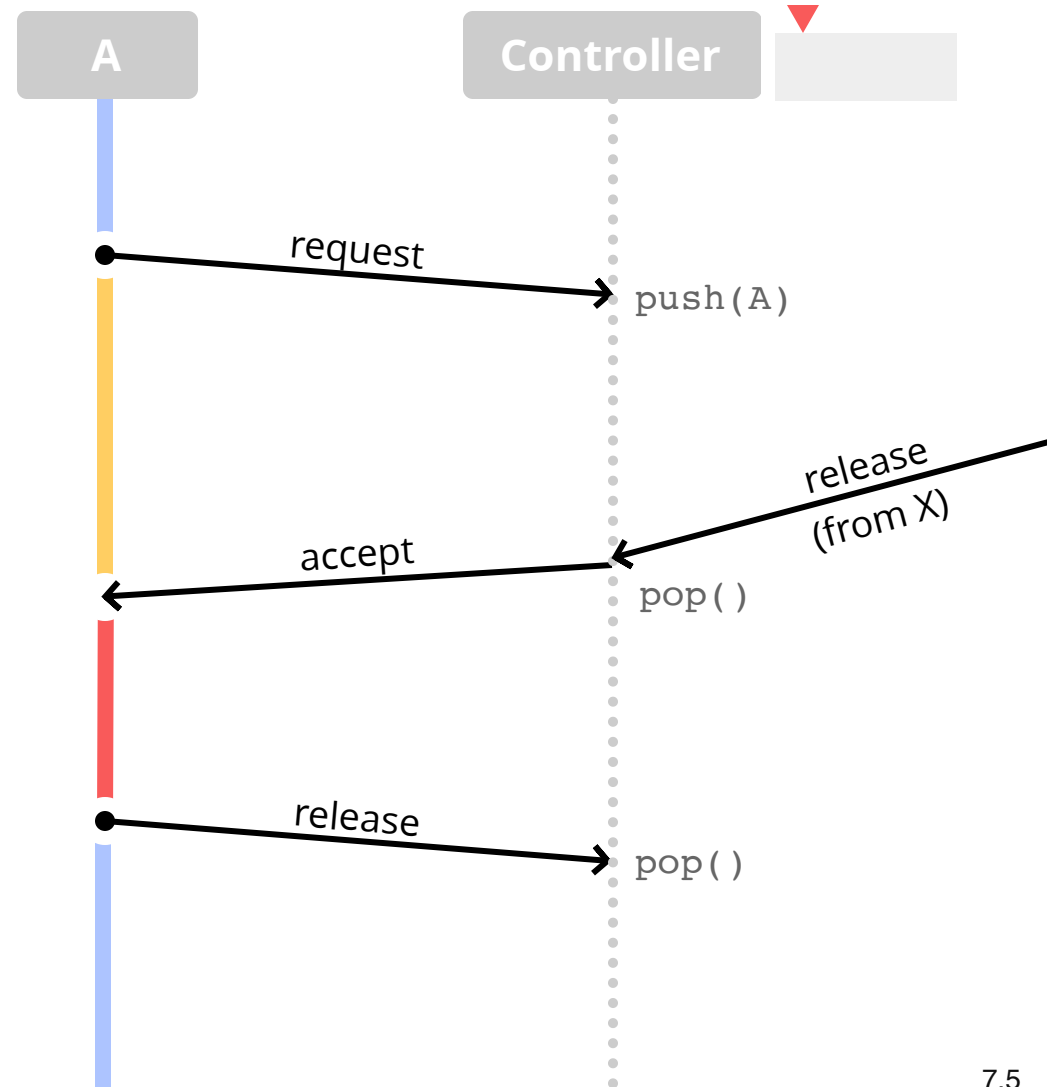
On request

Push demandeur dans la file

On release

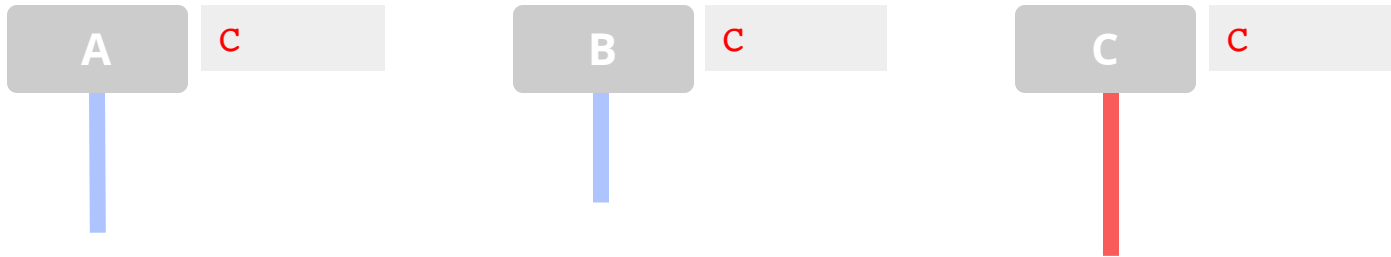
Pop tête de file

Envoi de accept à nouvelle tête



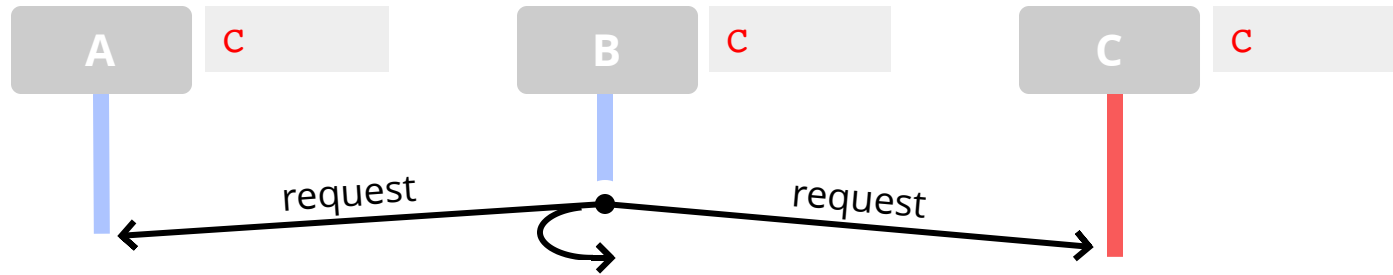
Solution répartie

Version naïve



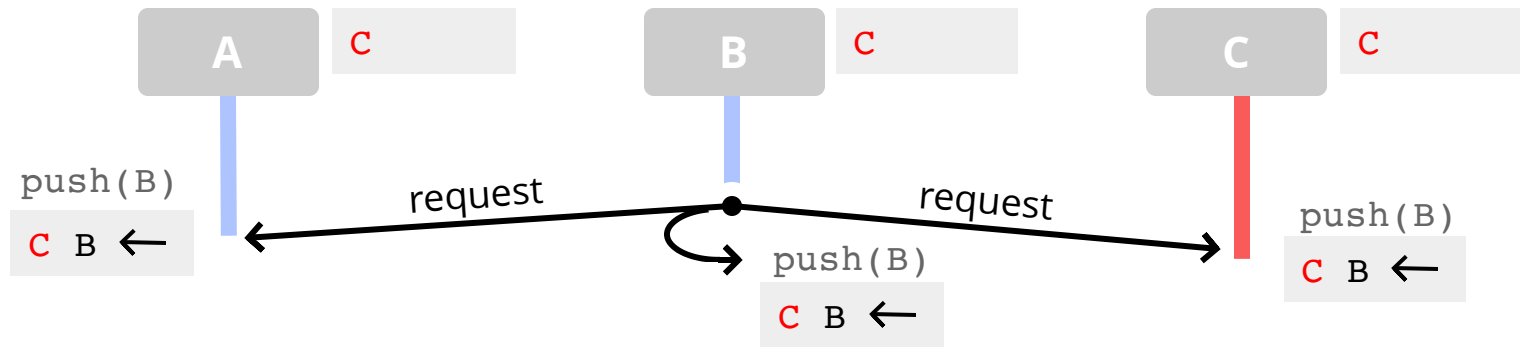
Solution répartie

Version naïve



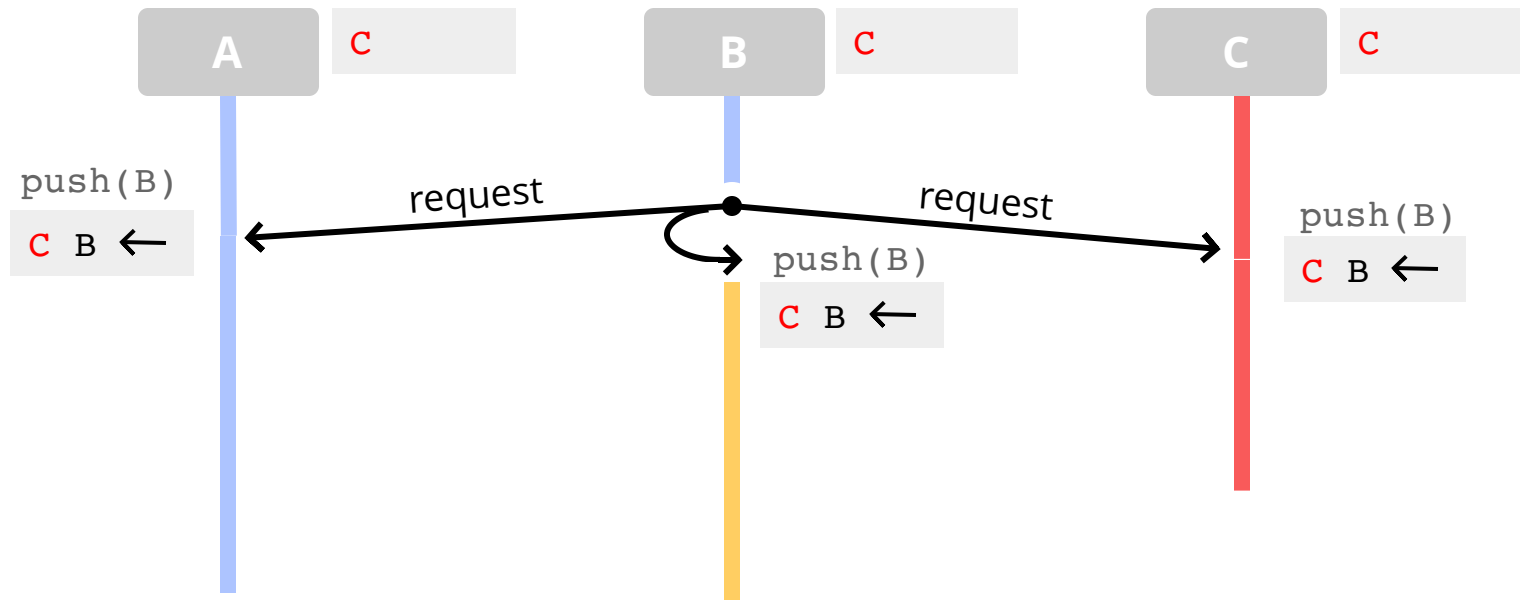
Solution répartie

Version naïve



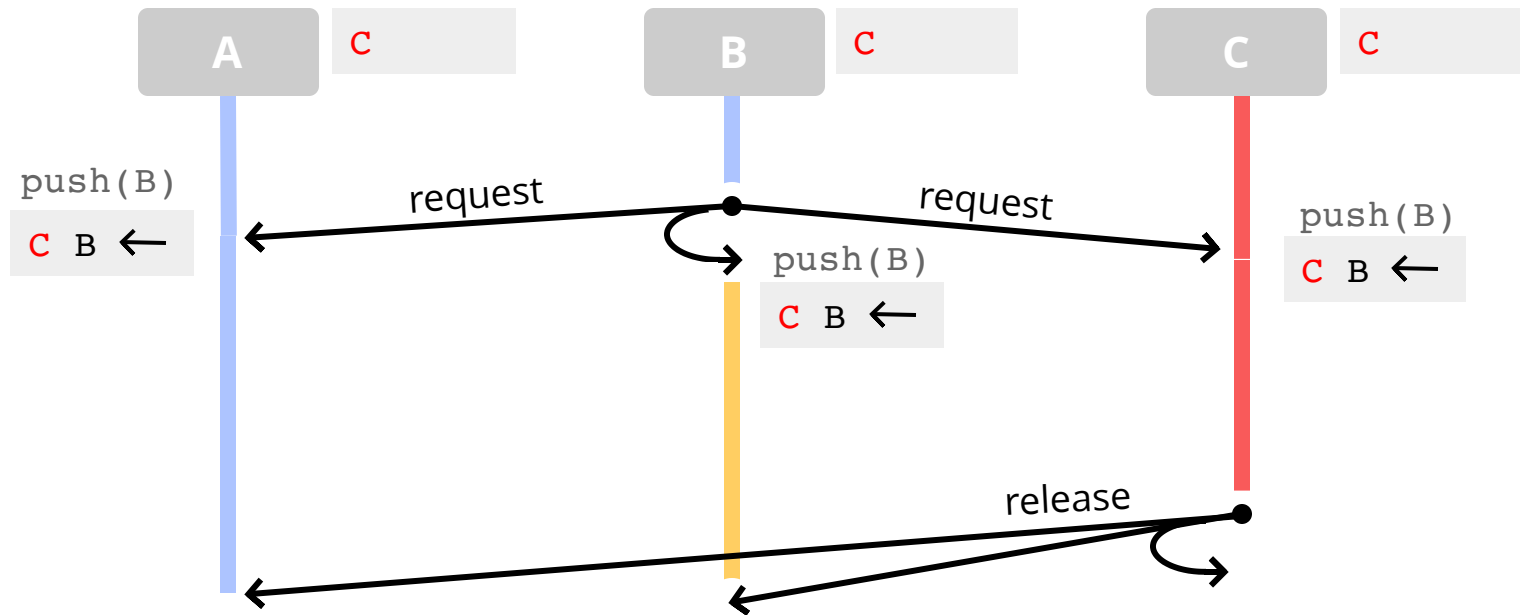
Solution répartie

Version naïve



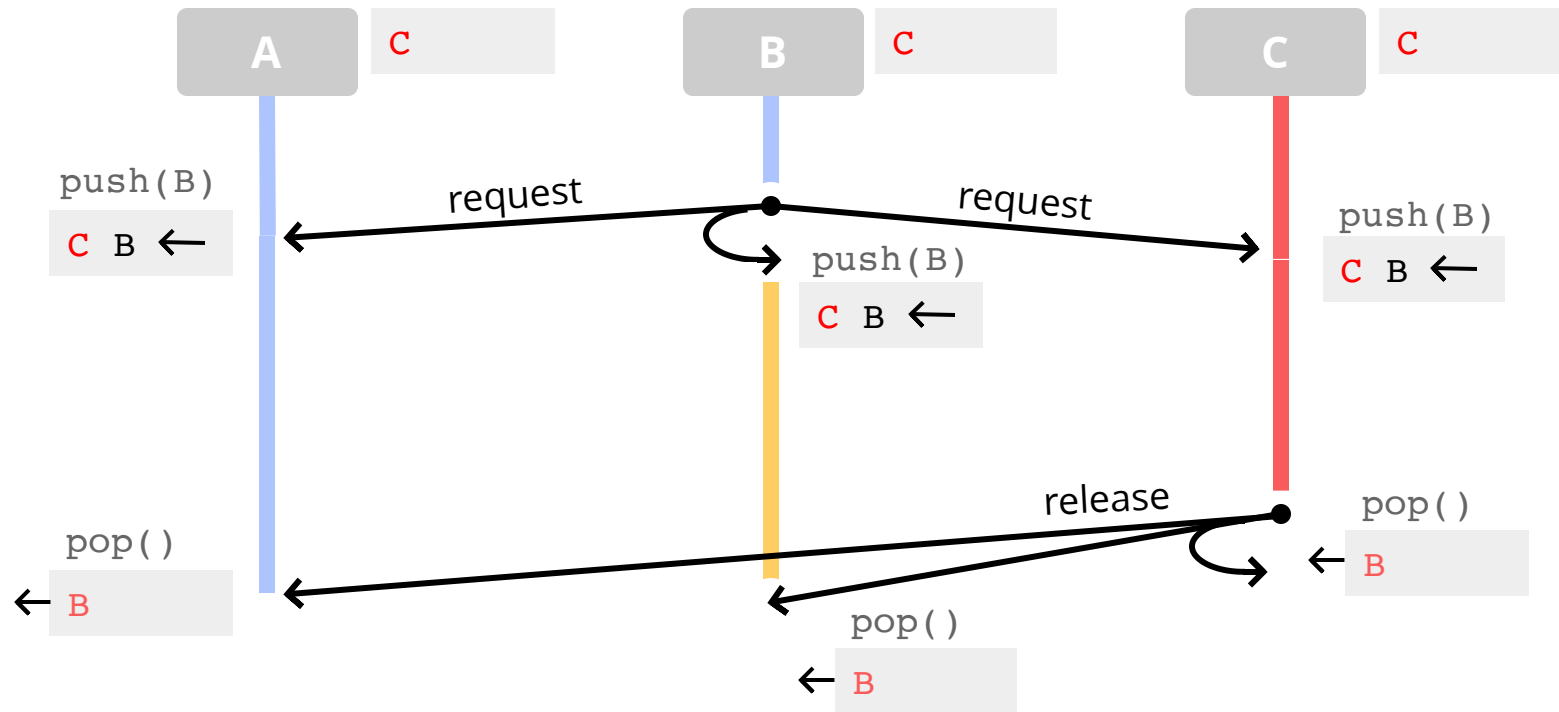
Solution répartie

Version naïve



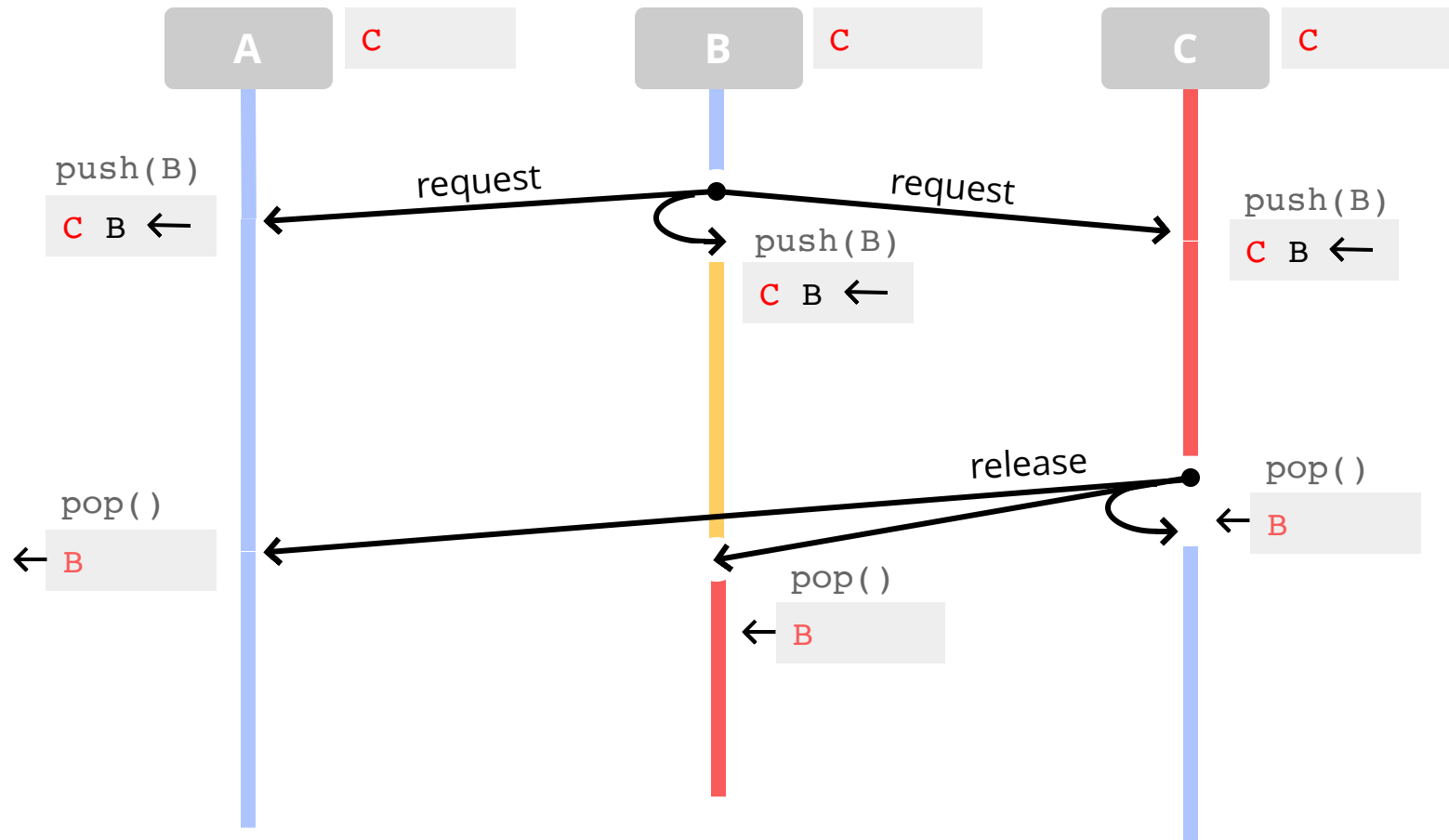
Solution répartie

Version naïve



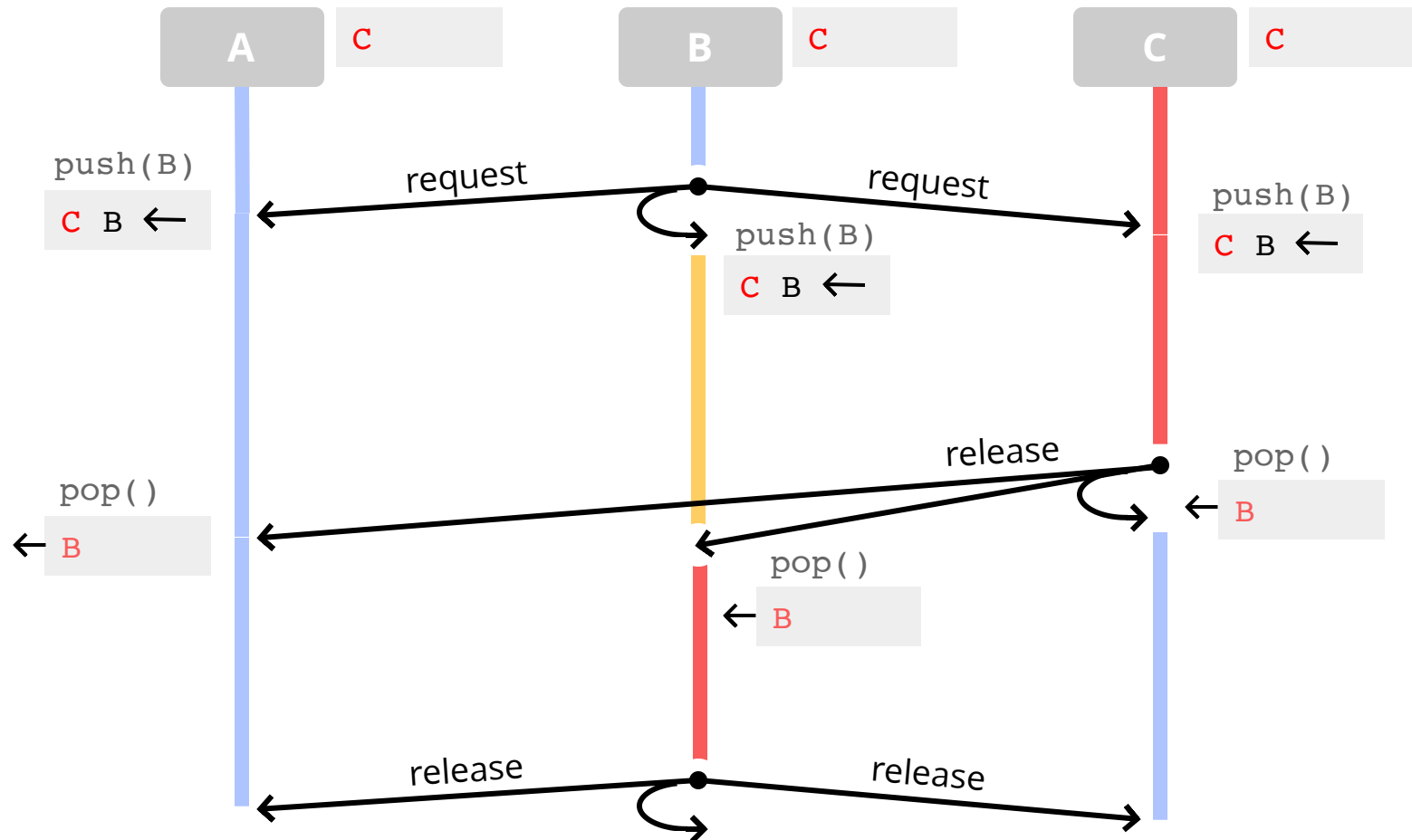
Solution répartie

Version naïve



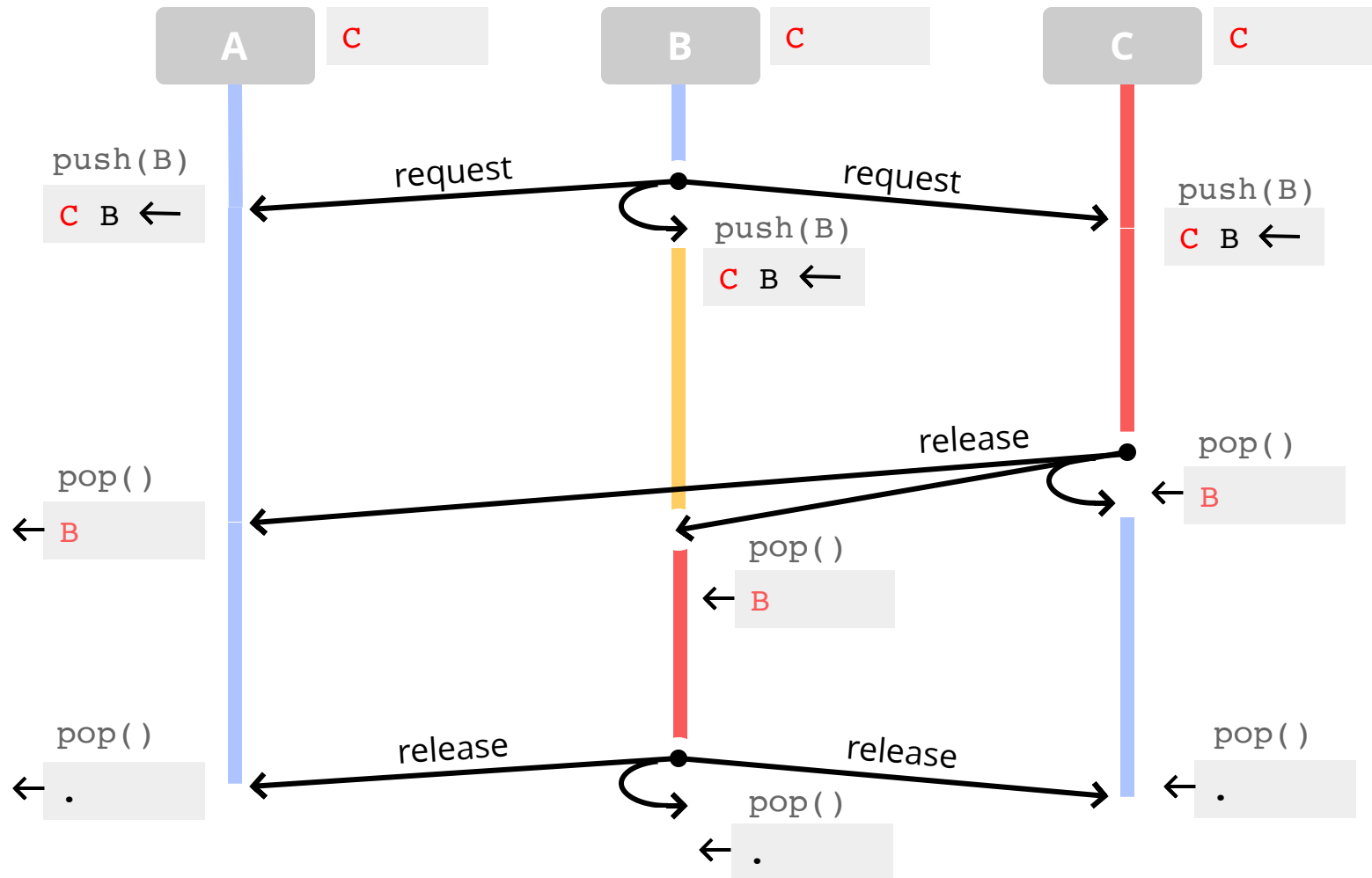
Solution répartie

Version naïve



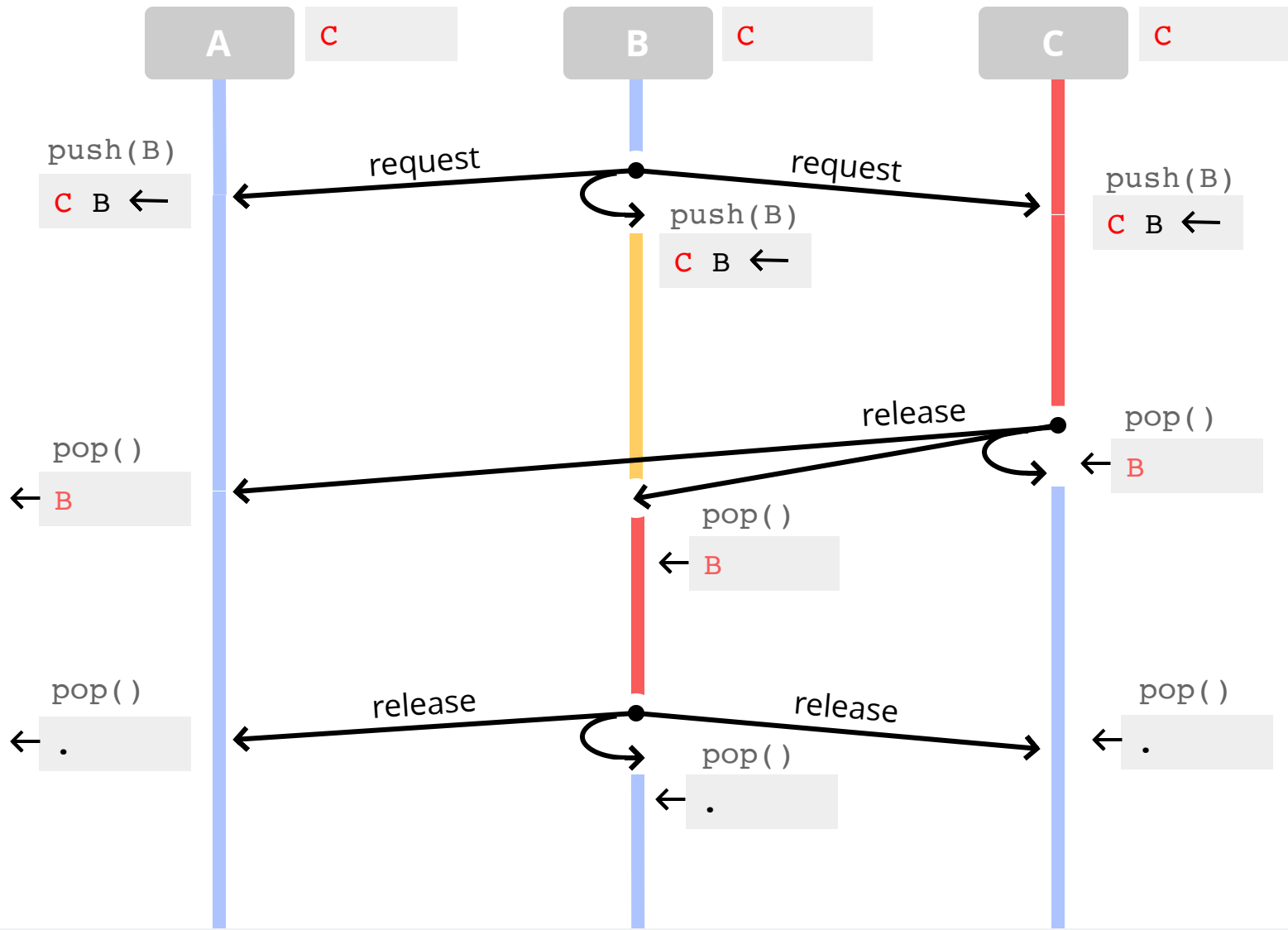
Solution répartie

Version naïve



Solution répartie

Version naïve



Solution répartie

Version naïve - Résumé

La tête de la file est toujours celle actuellement en SC.

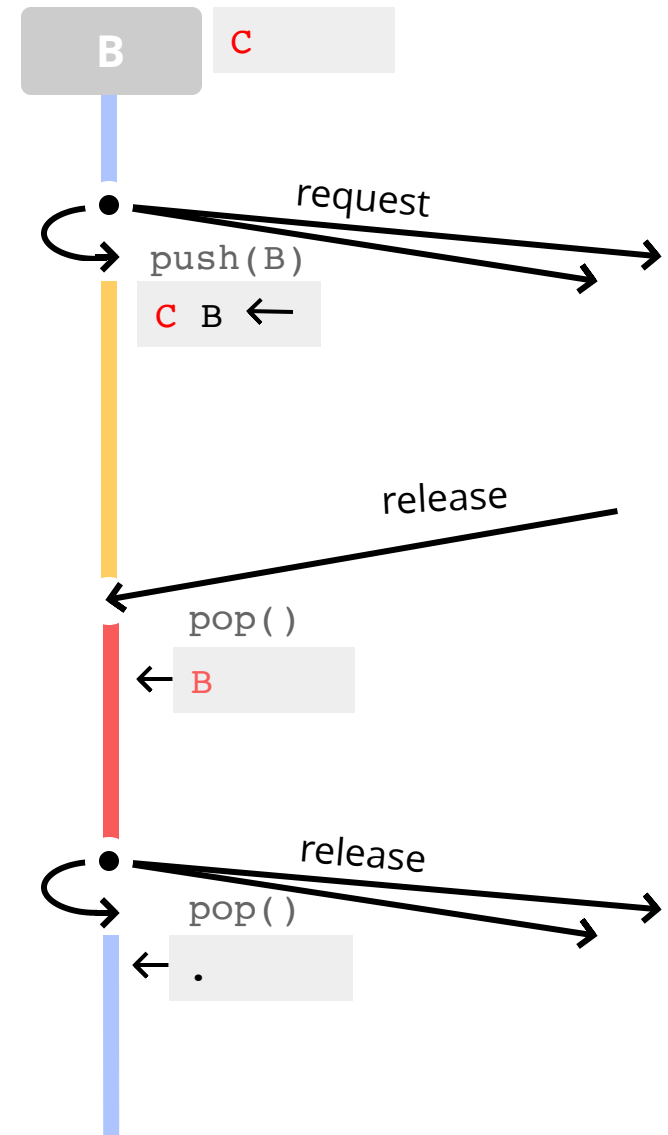
On request

Push demandeur dans la file

On release

Pop tête de file

Si je suis nouvelle tête, entrer en SC.



Solution répartie

Version naïve - Résumé

La tête de la file est toujours celle actuellement en SC.

On request

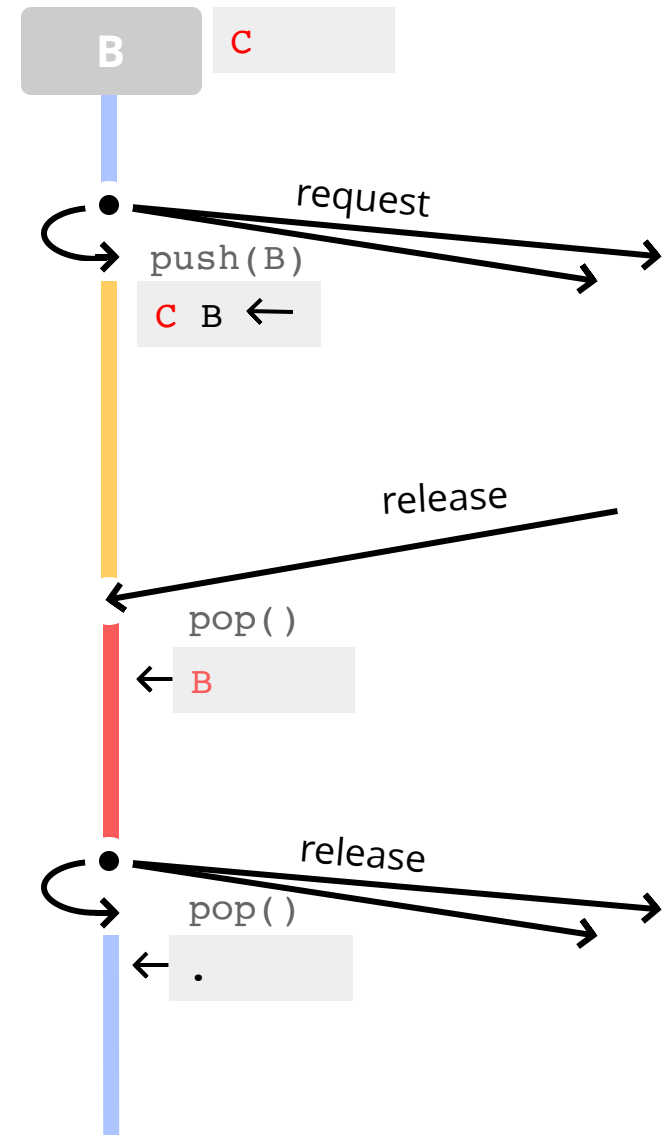
Push demandeur dans la file

On release

Pop tête de file

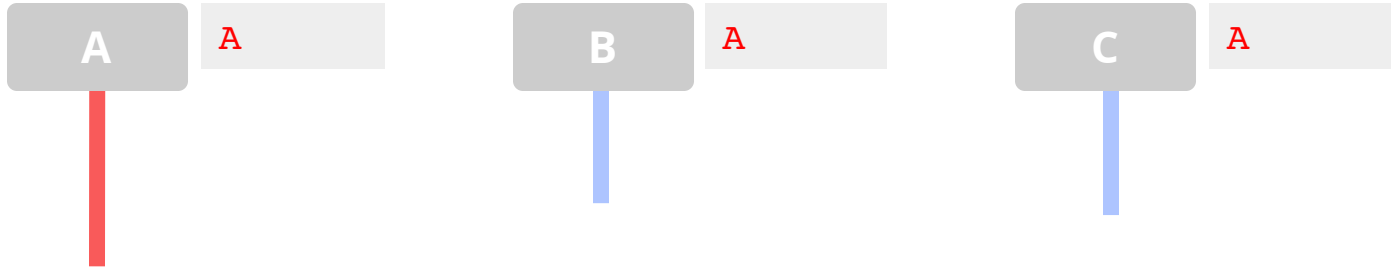
Si je suis nouvelle tête, entrer en SC.

Problème ?



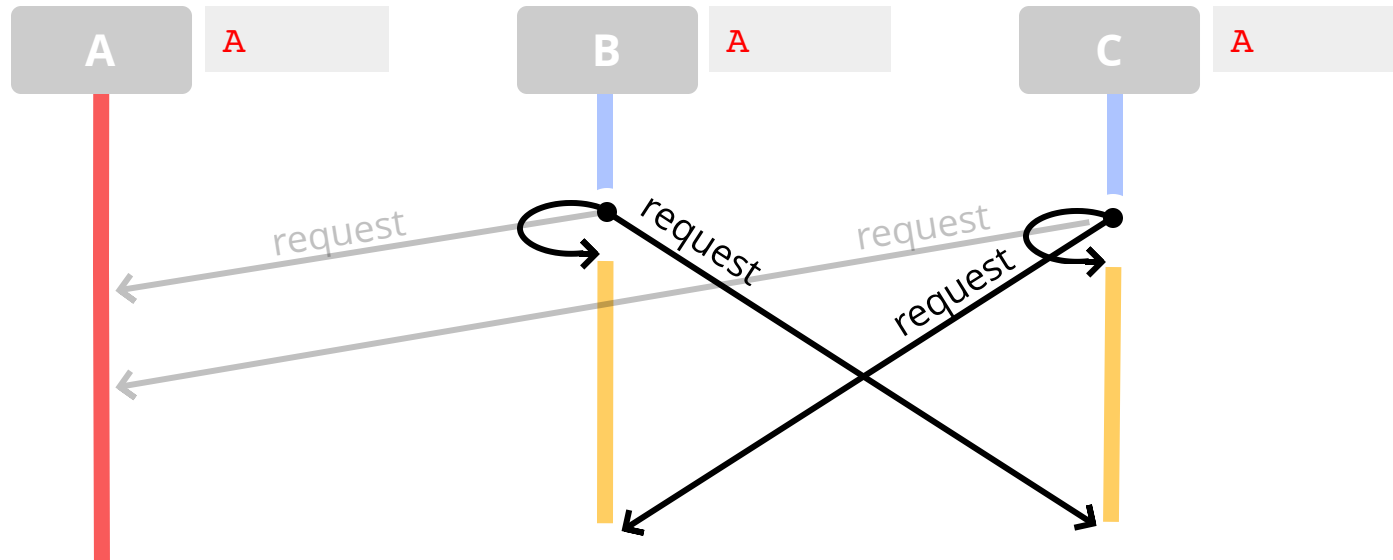
Solution répartie

Version naïve - Problème



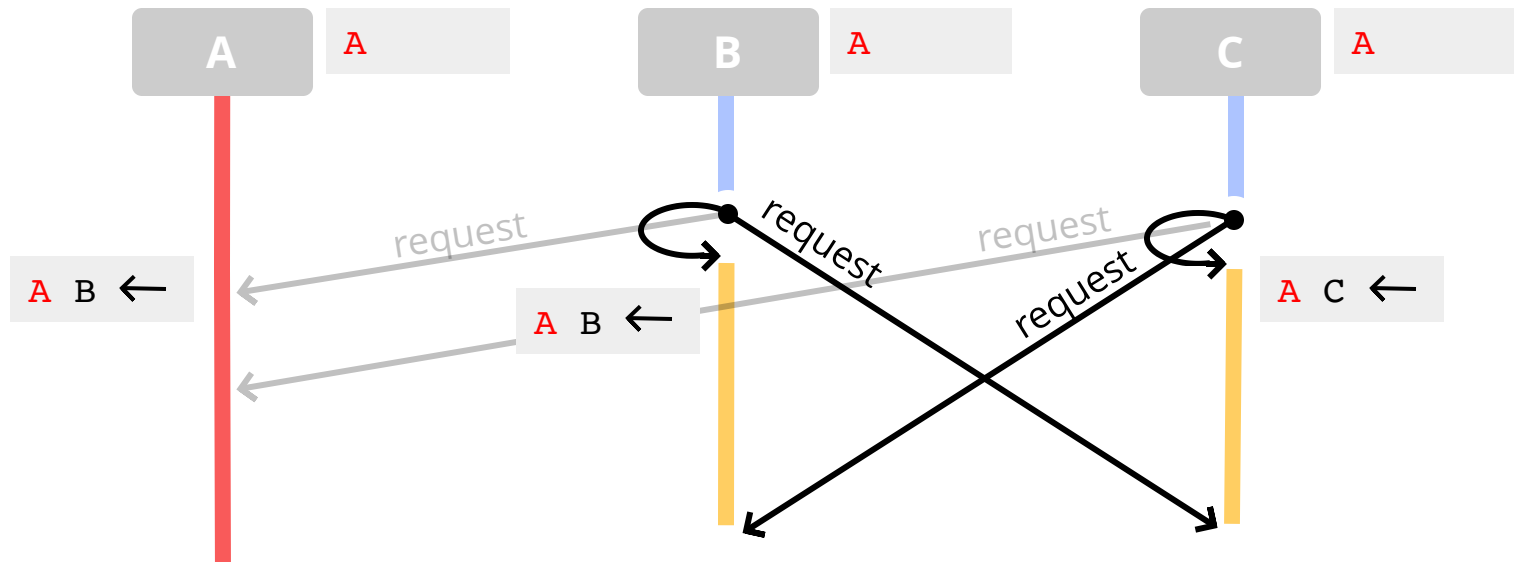
Solution répartie

Version naïve - Problème



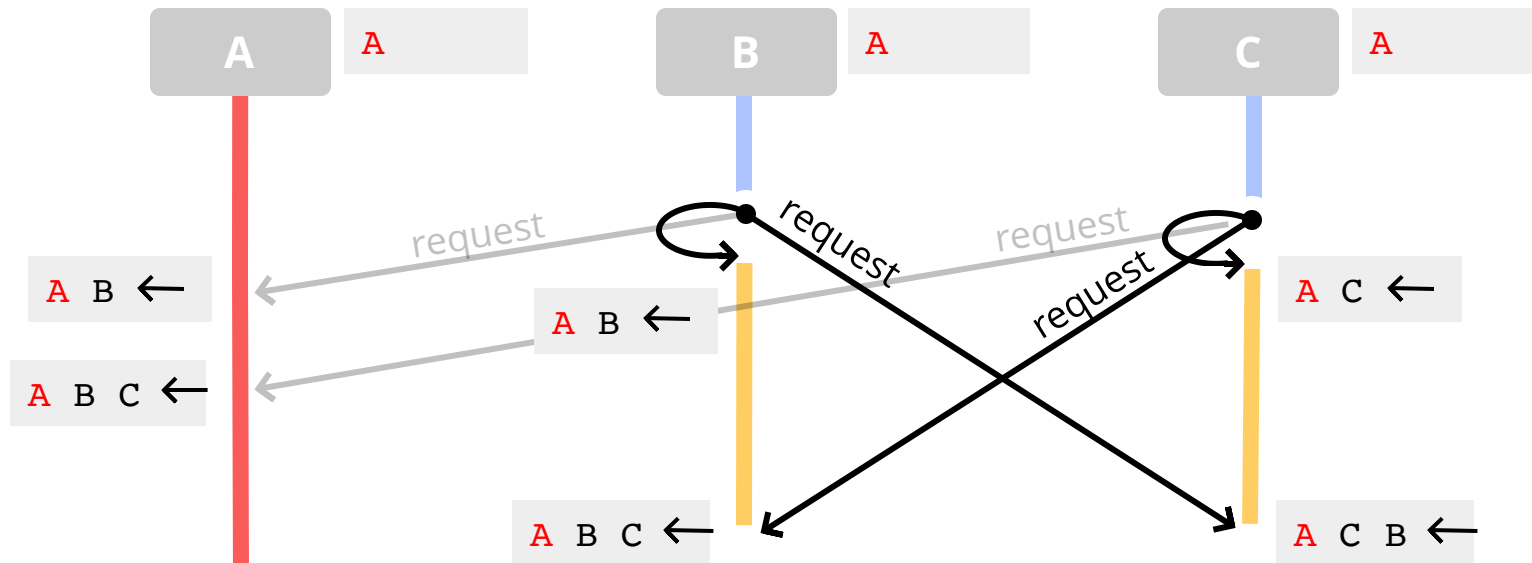
Solution répartie

Version naïve - Problème



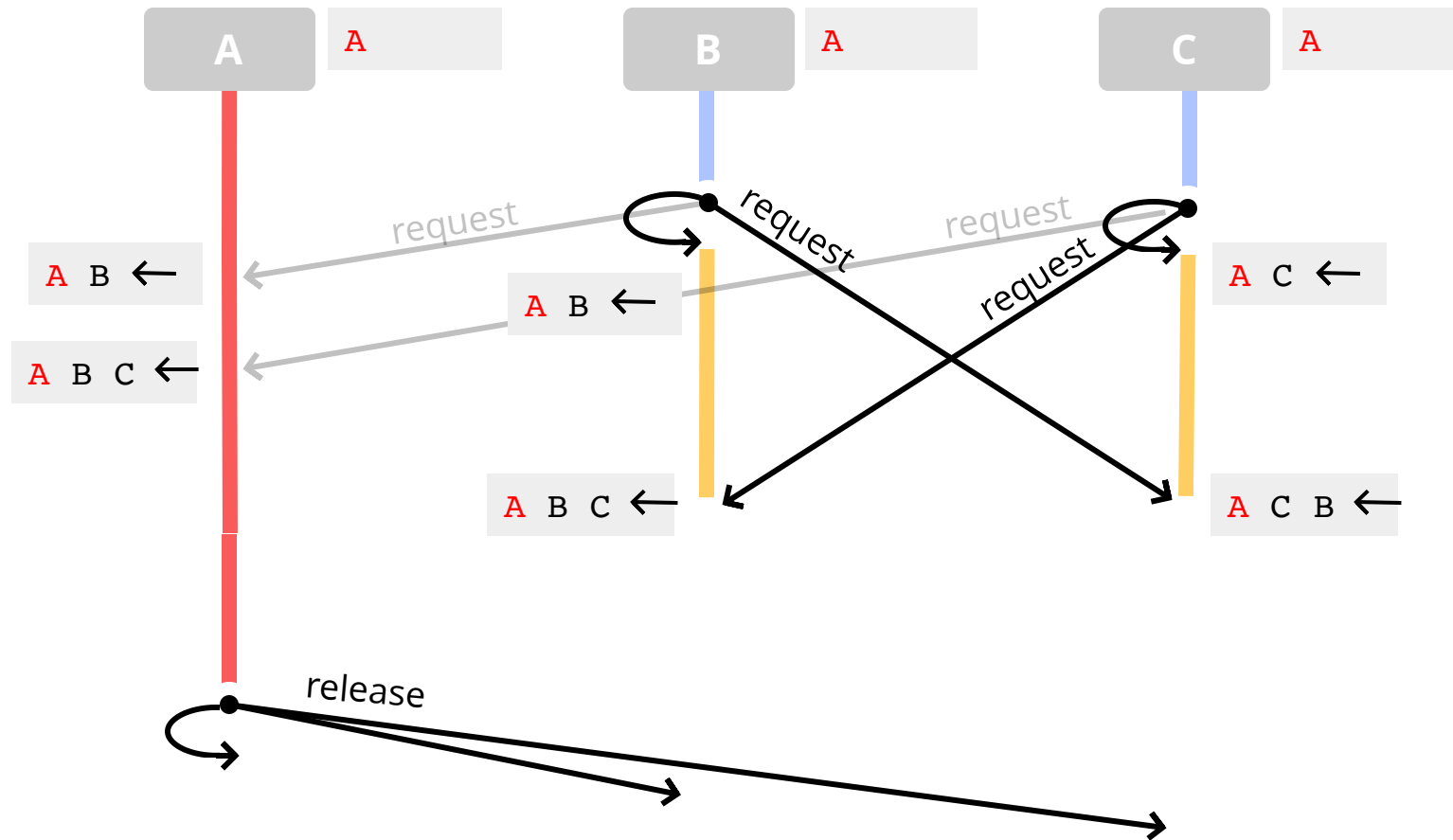
Solution répartie

Version naïve - Problème



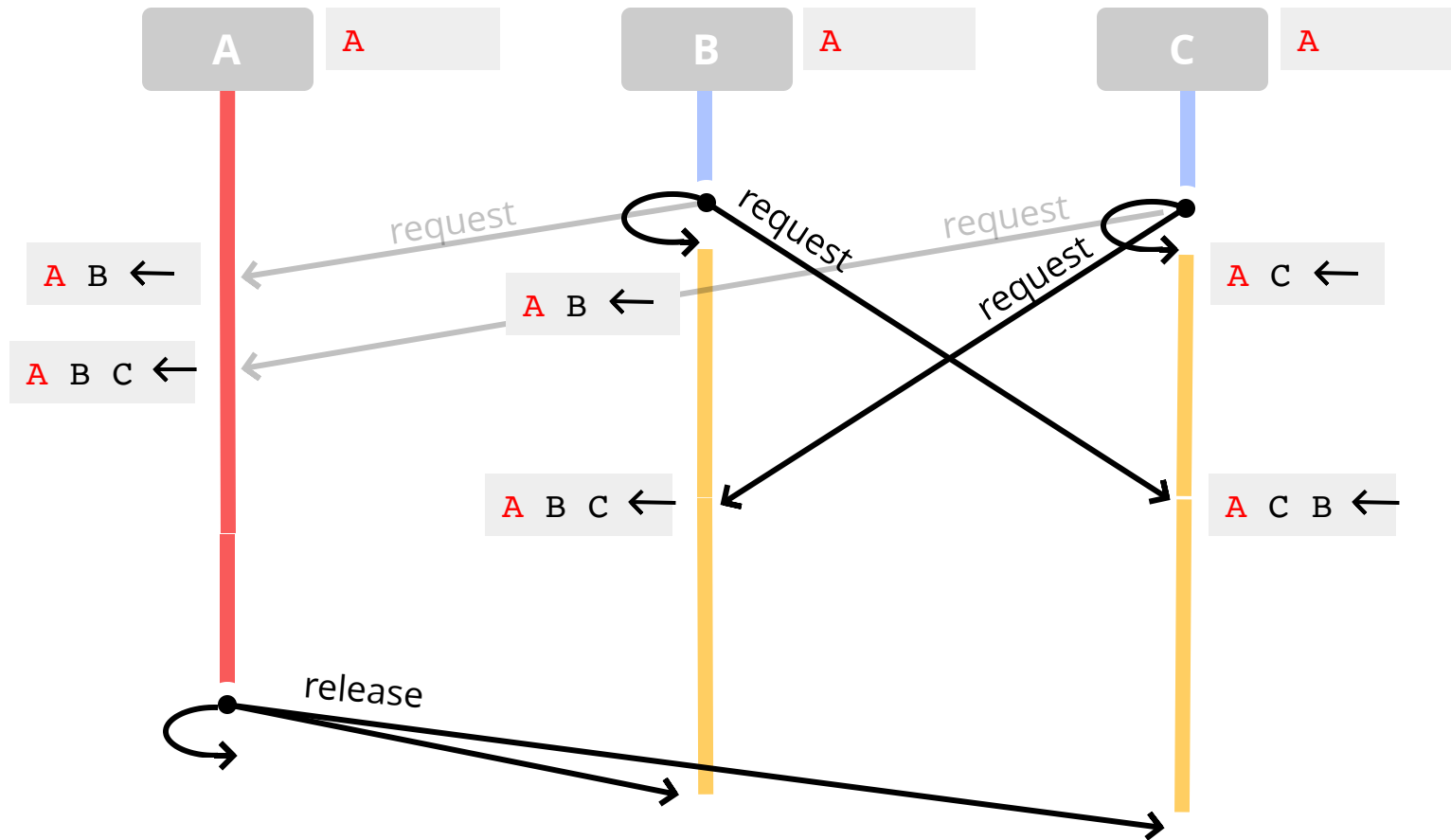
Solution répartie

Version naïve - Problème



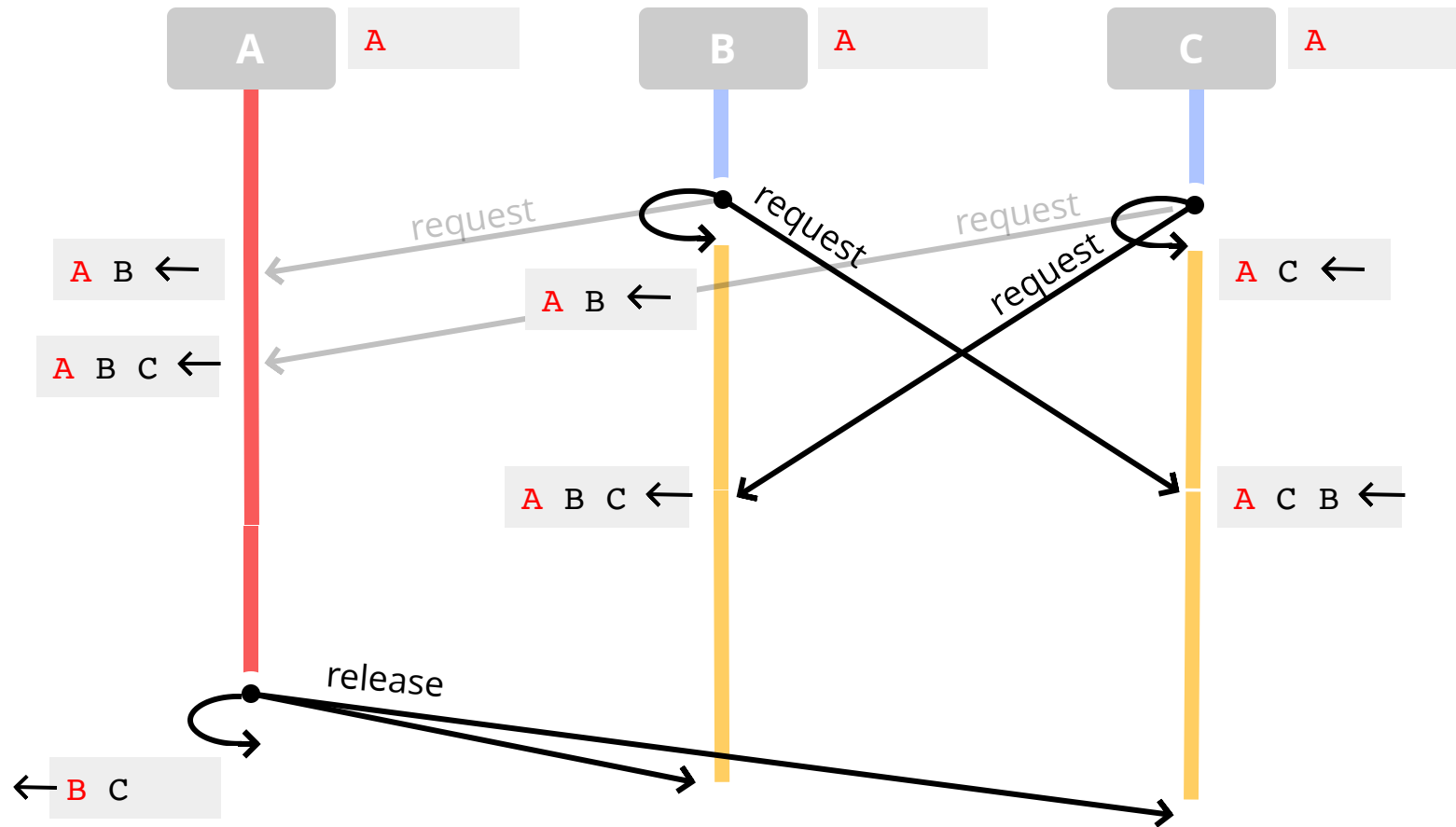
Solution répartie

Version naïve - Problème



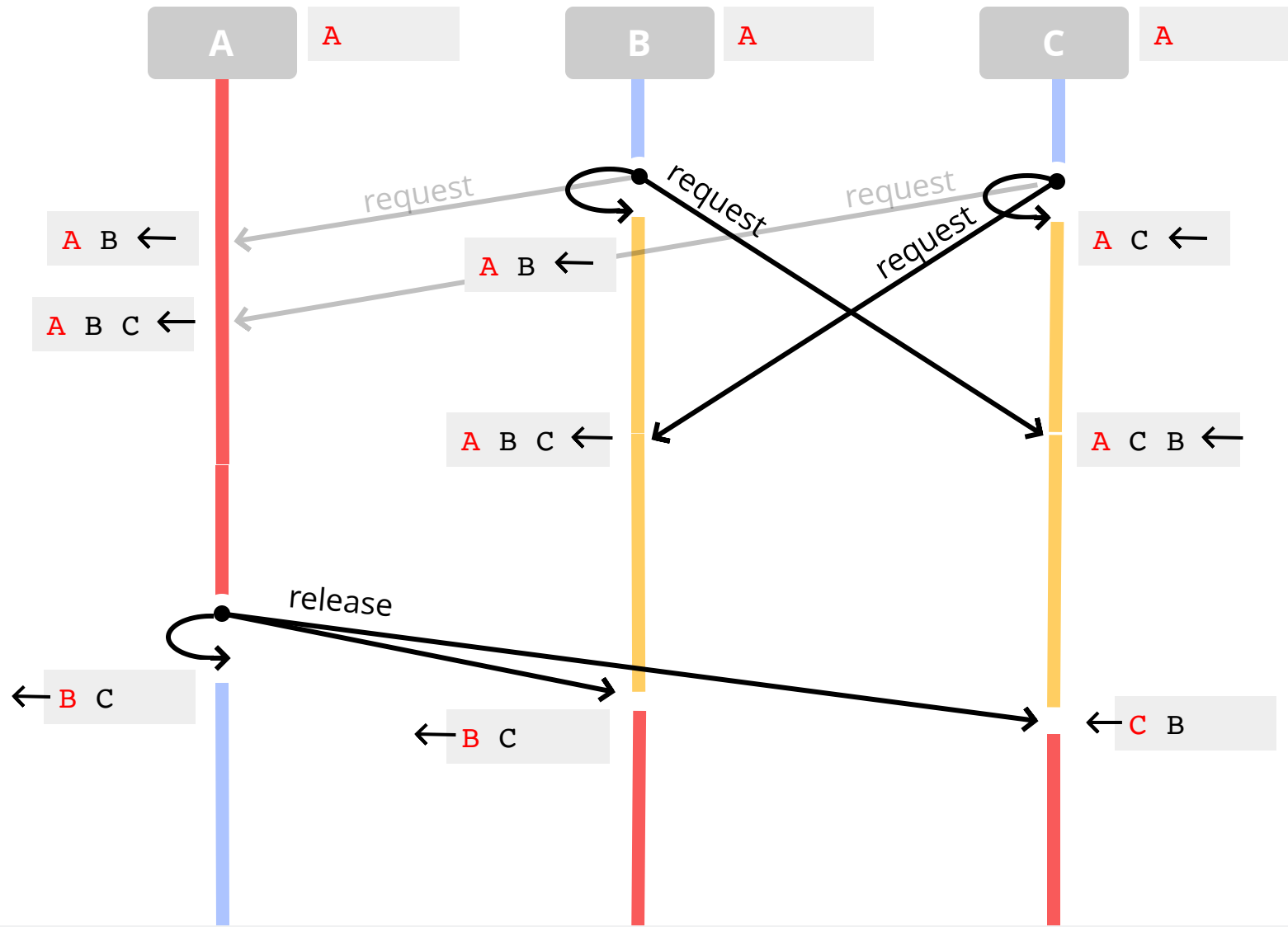
Solution répartie

Version naïve - Problème



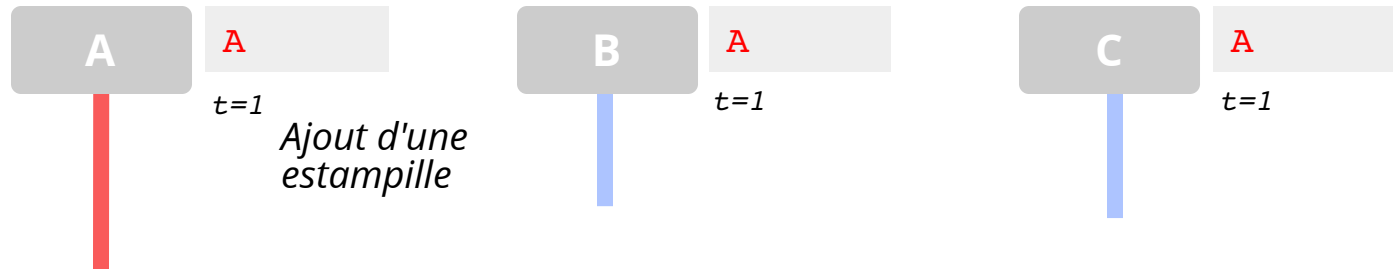
Solution répartie

Version naïve - Problème



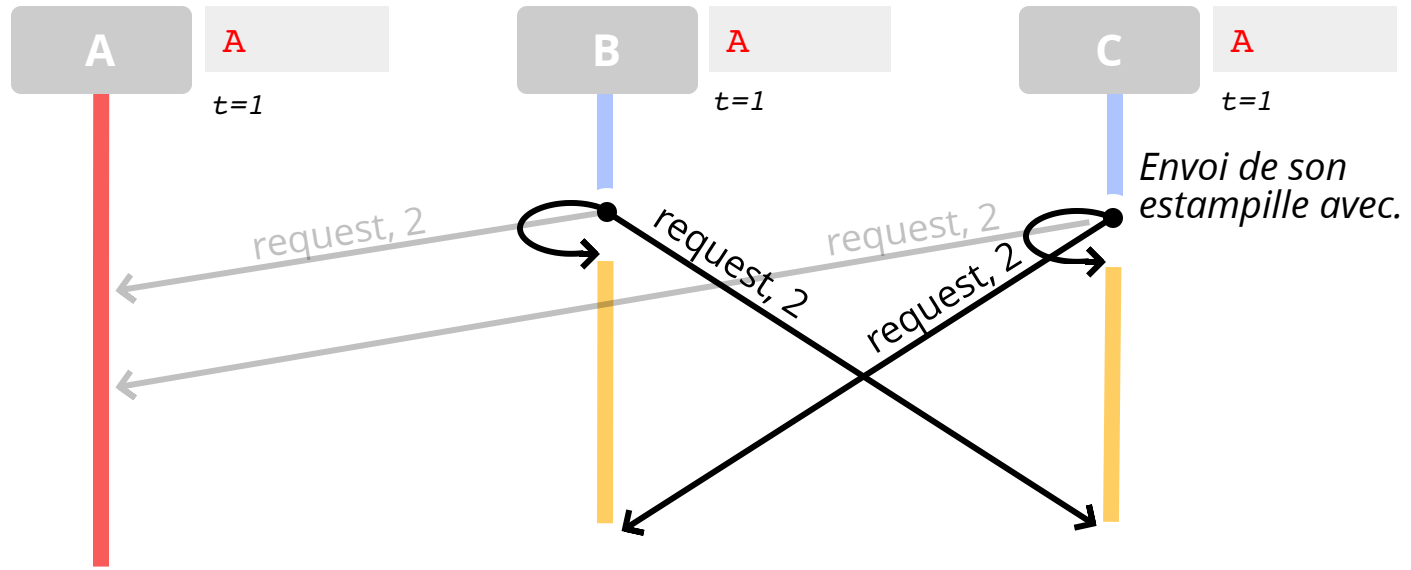
Solution répartie

Version naïve *corrigée*



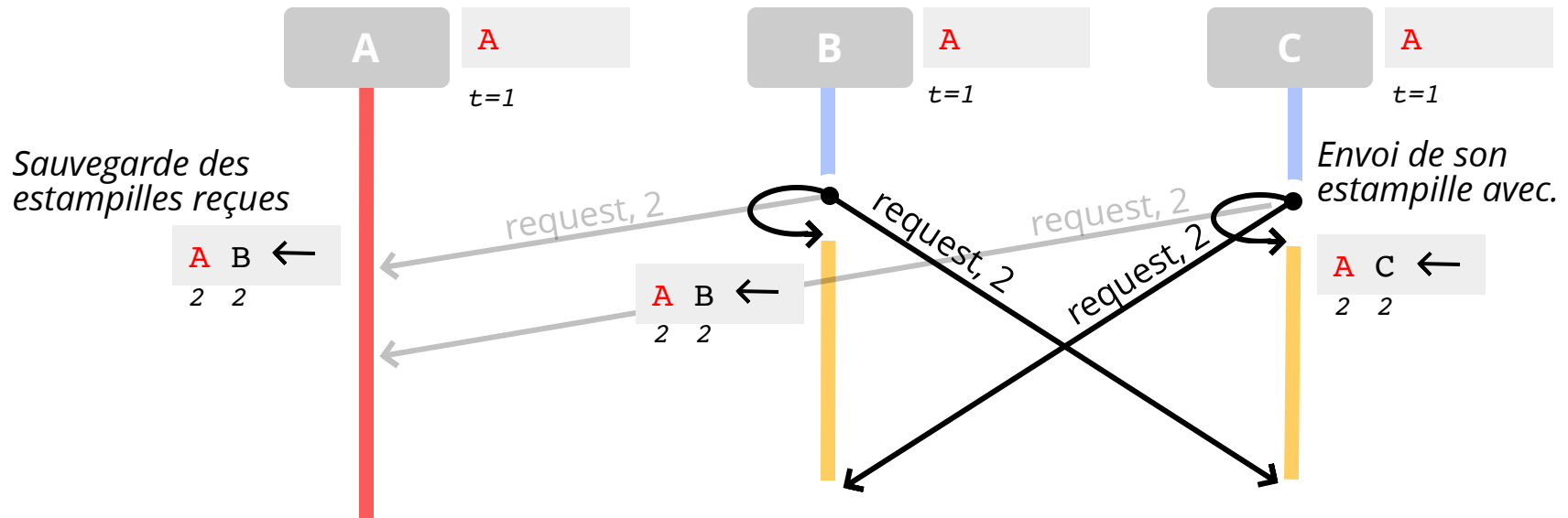
Solution répartie

Version naïve *corrigée*



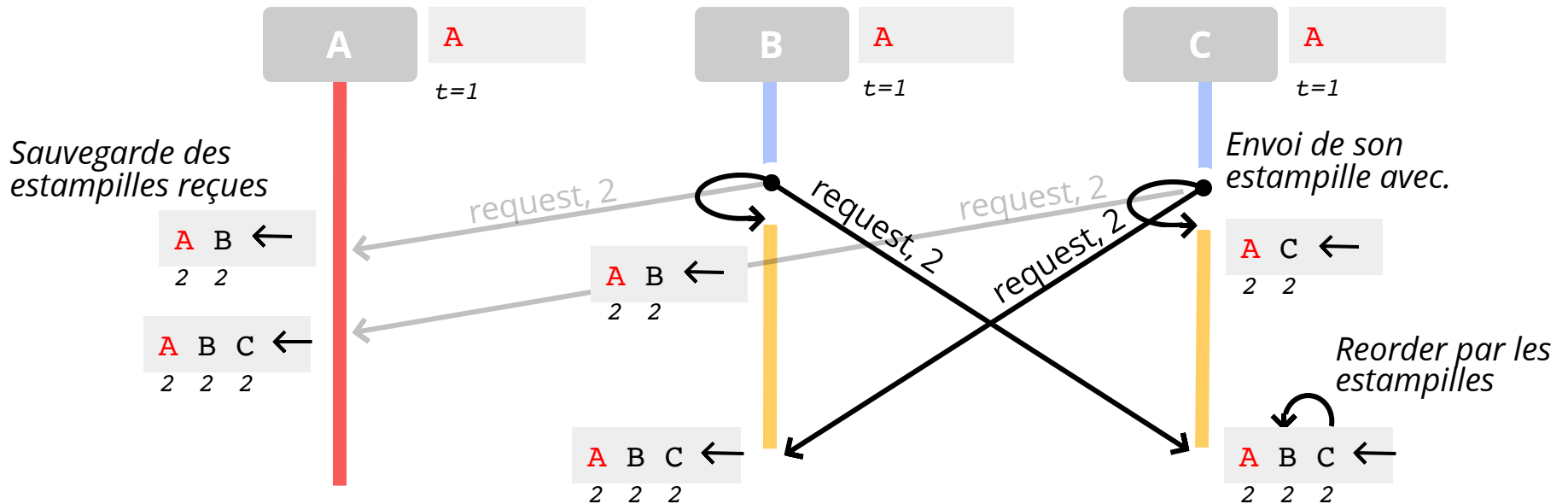
Solution répartie

Version naïve *corrigée*



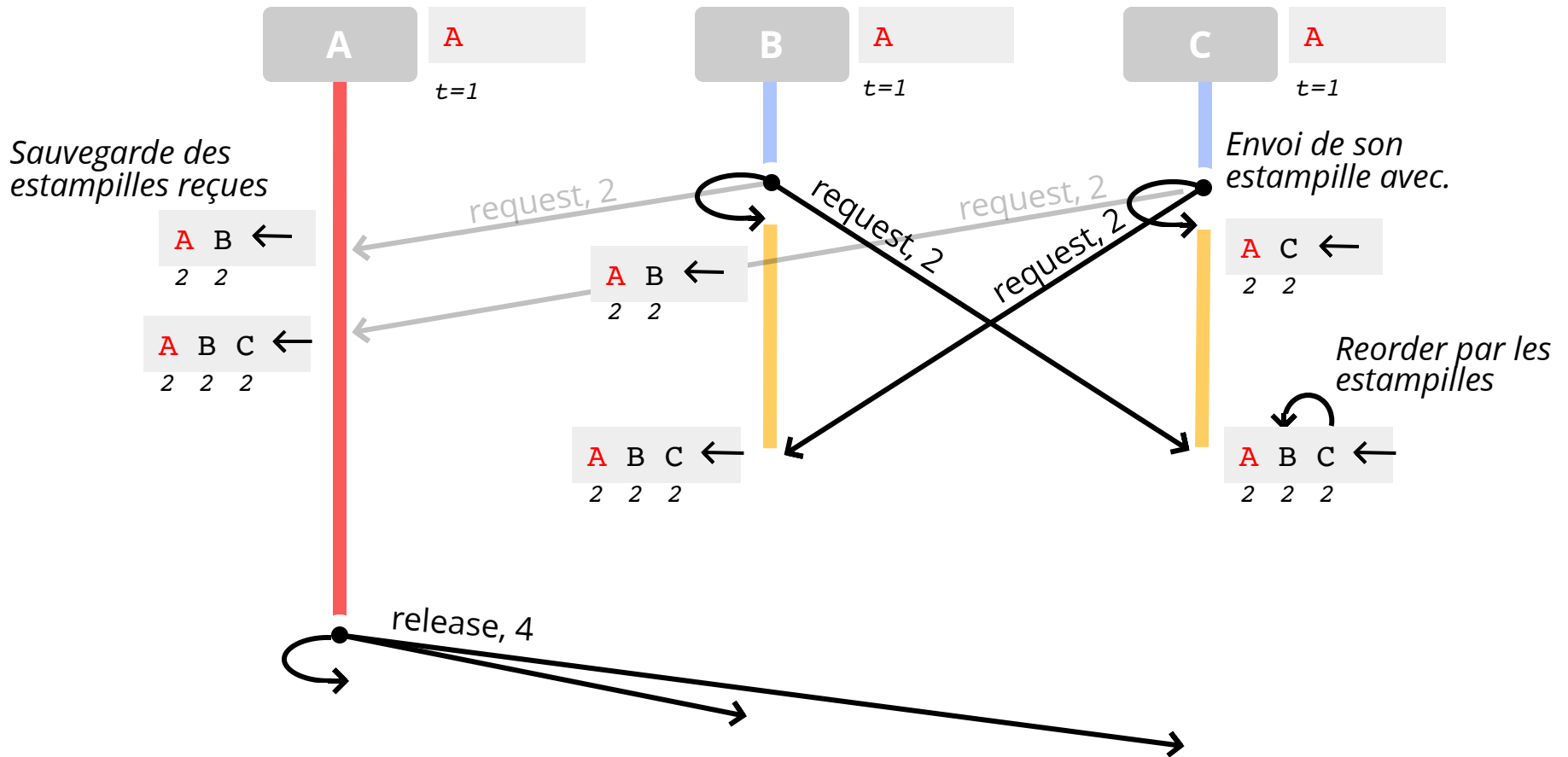
Solution répartie

Version naïve *corrigée*



Solution répartie

Version naïve *corrigée*

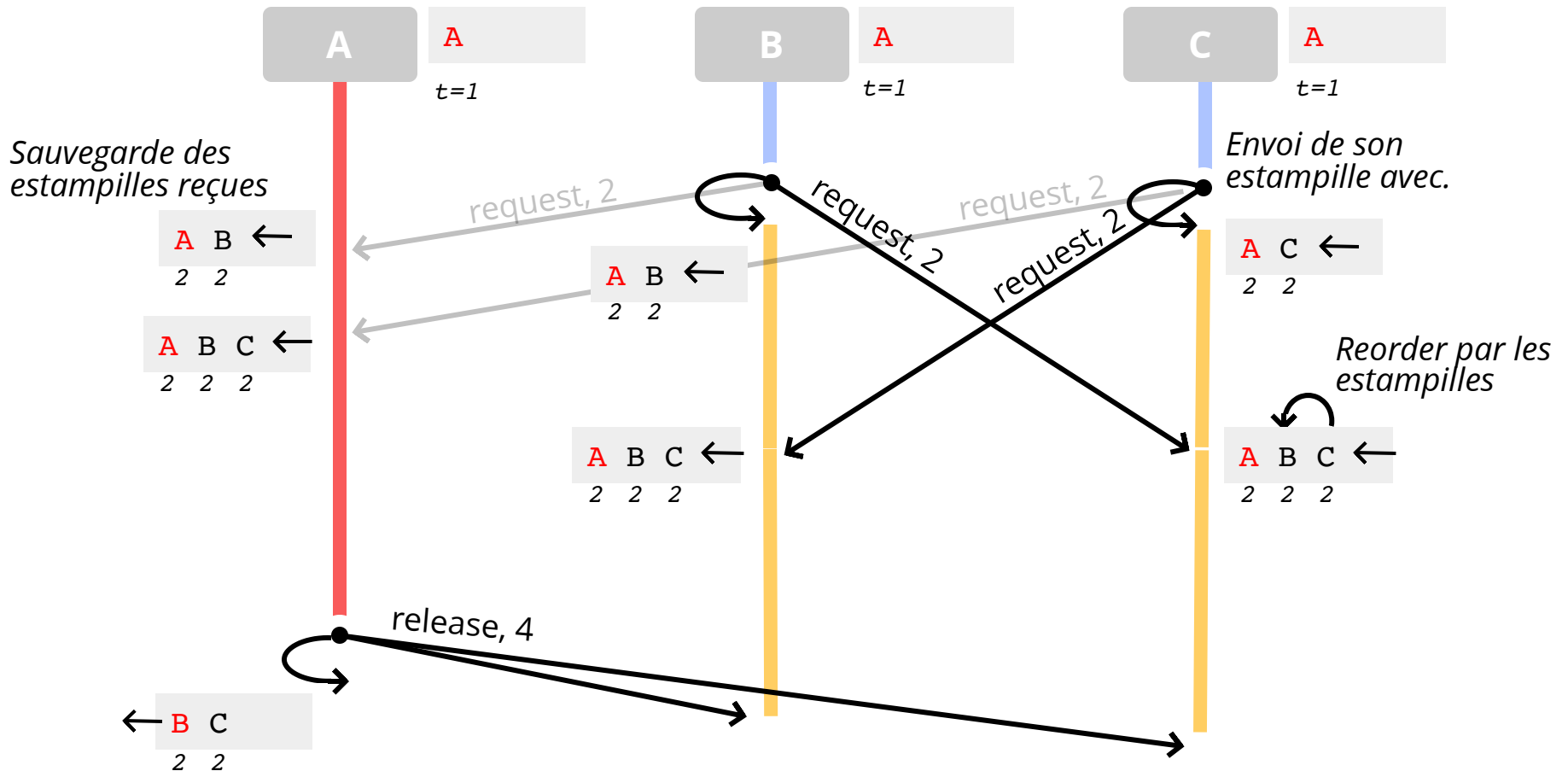


Version naïve corrigée



Solution répartie

Version naïve *corrigée*

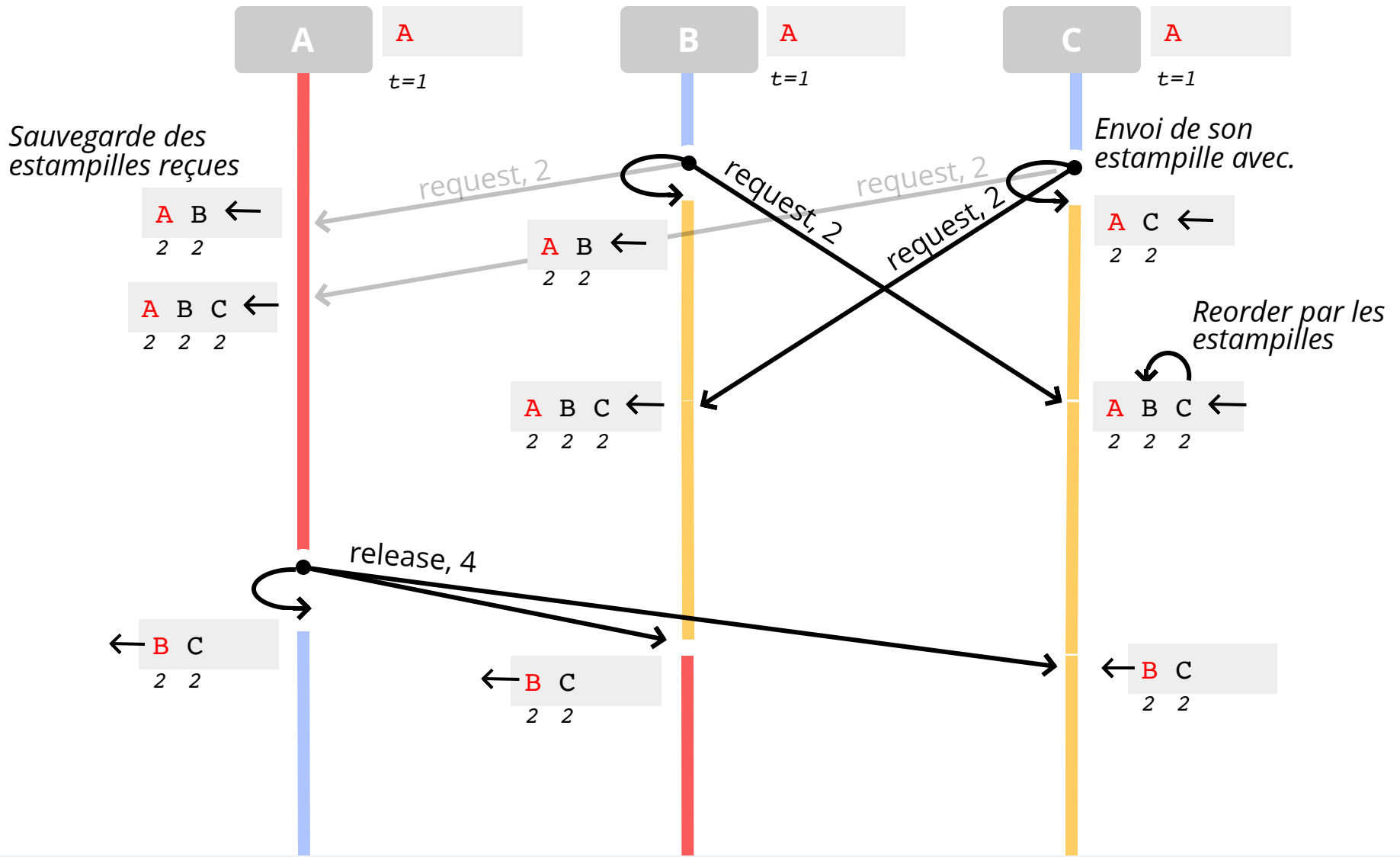


Version naïve corrigée



Solution répartie

Version naïve *corrigée*



Solution répartie

Version naïve corrigée - Résumé

La tête de la file est toujours celle actuellement en SC.

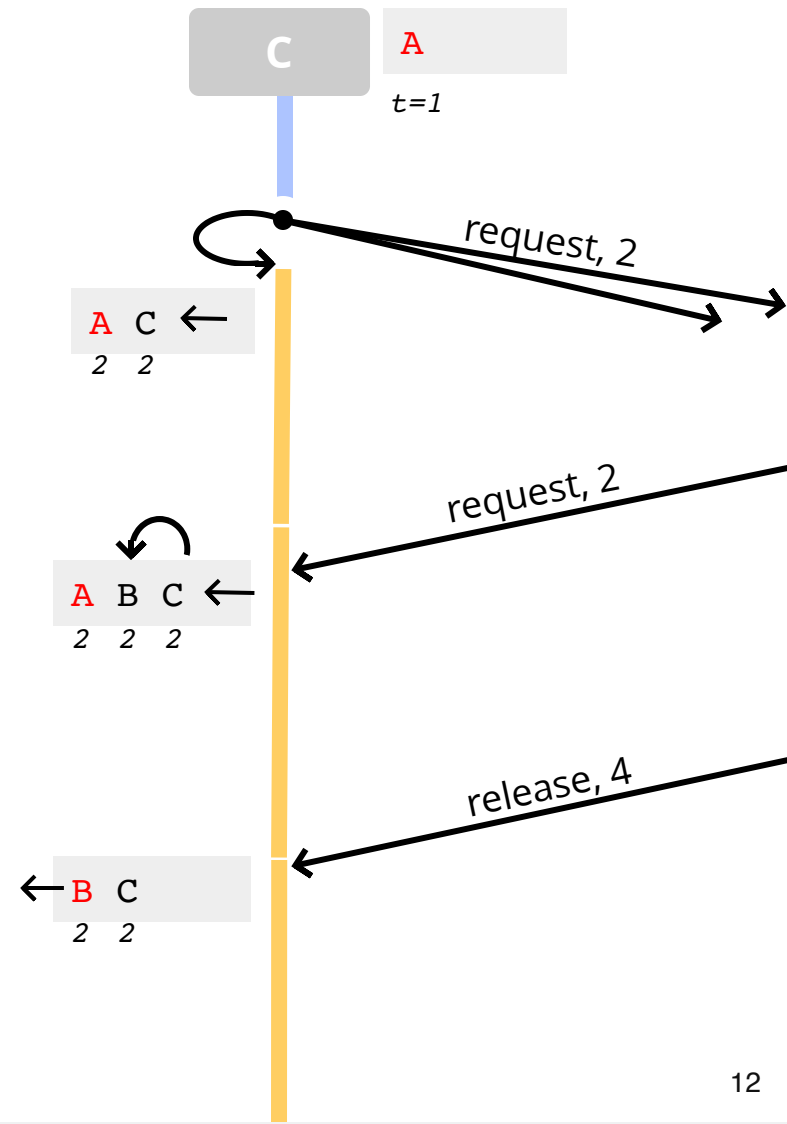
On request

Push demandeur dans la file

On release

Pop tête de file

Si je suis nouvelle tête, entrer en SC.



Solution répartie

Version naïve corrigée - Résumé

La tête de la file est toujours celle actuellement en SC.

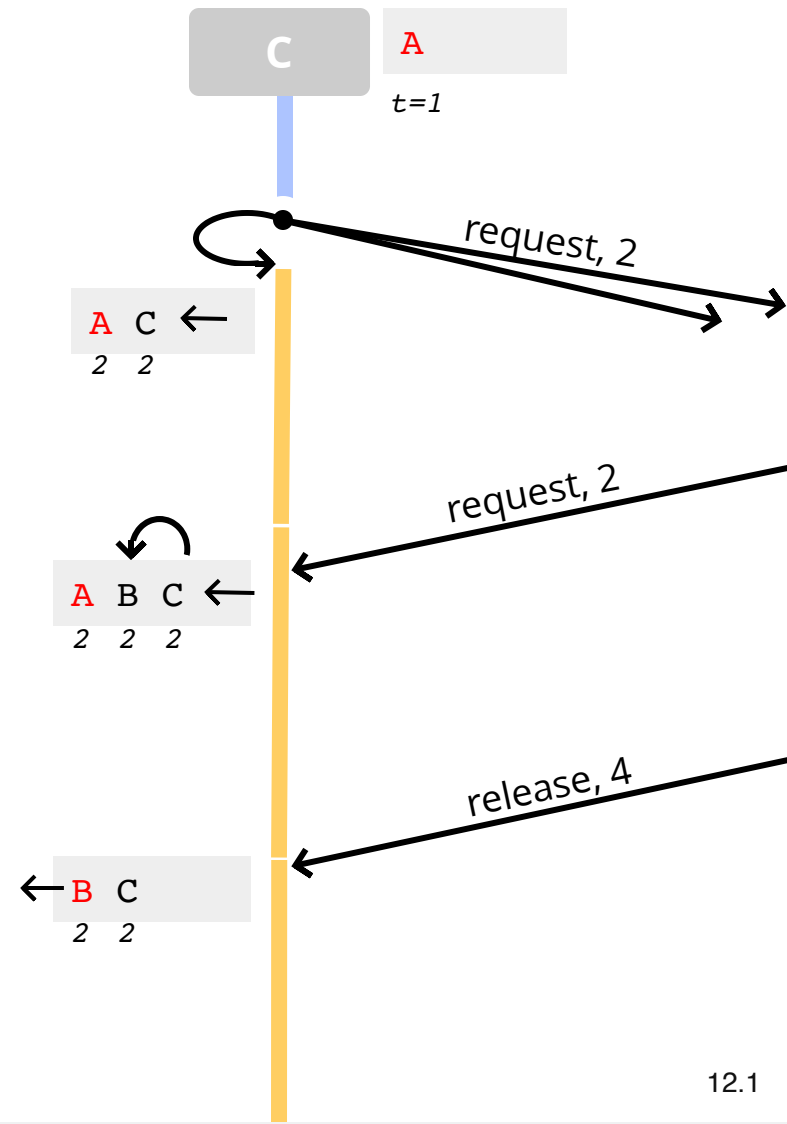
On request

Push demandeur dans la file avec son timestamp.

On release

Pop tête de file

Si je suis nouvelle tête, entrer en SC.



Solution répartie

Version naïve corrigée - Résumé

La tête de la file est toujours celle actuellement en SC.

On request

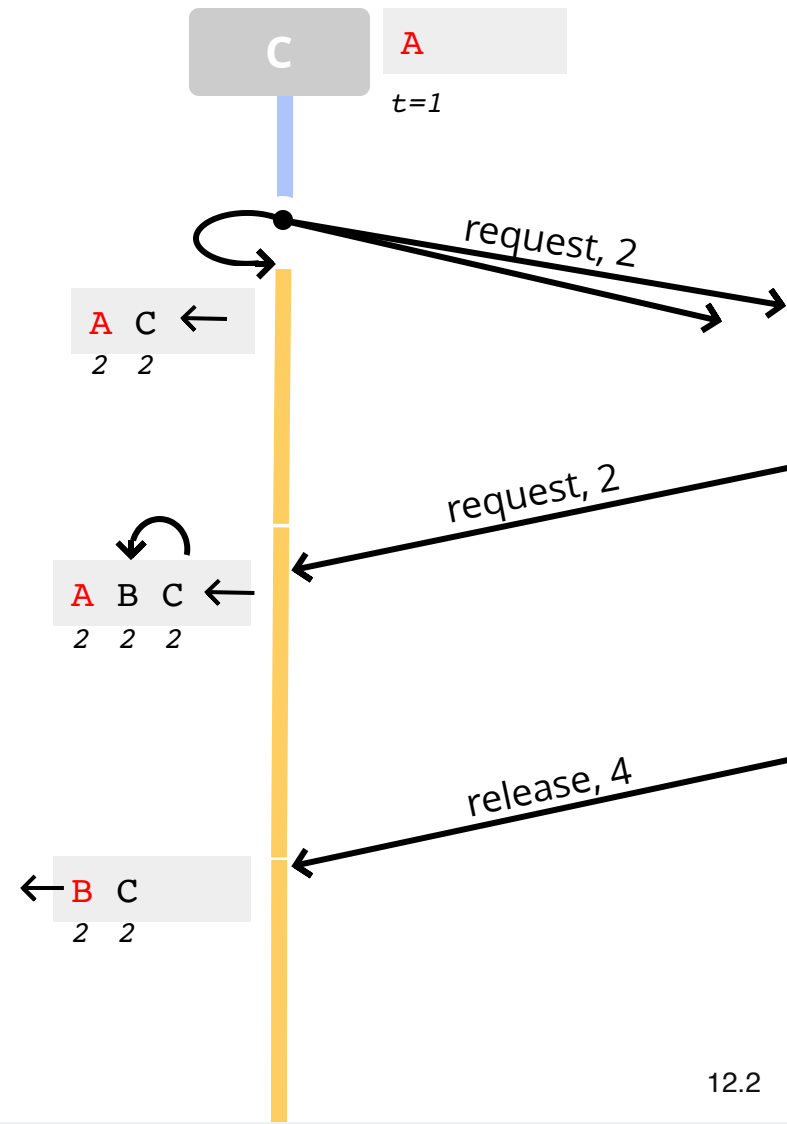
Push demandeur dans la file avec son timestamp.

Trie la file par heure de Lamport.
(avec résolution arbitraire des égalités)

On release

Pop tête de file

Si je suis nouvelle tête, entrer en SC.



Version naïve corrigée - Résumé

On request

Trie la file par heure de Lamport.
(avec résolution arbitraire des égalités)

Pop tête de file

Si je suis nouvelle tête, entrer en SC.

The diagram illustrates a priority queue simulation. A vertical yellow line represents the timeline. Three processes are shown as gray boxes with their names and priority values (2) below them:

- Process C:** Located at the top, with a blue vertical bar extending downwards from its bottom edge.
- Process A:** Located in the middle, with a red 'A' and a black 'C' to its left, and two '2's below.
- Process B:** Located at the bottom, with a red 'B' and a black 'C' to its left, and two '2's below.

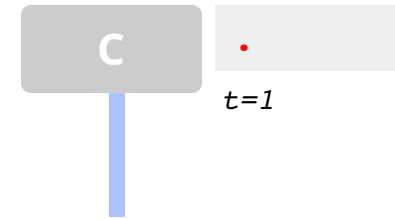
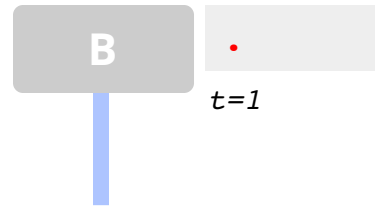
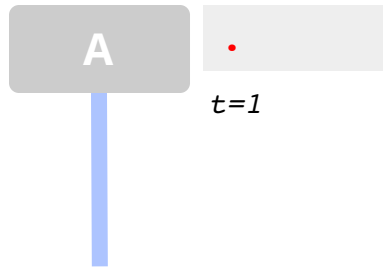
Arrows and labels indicate events on the timeline:

- A curved arrow points from the top of the timeline to the right, labeled "request, 2".
- A straight arrow points from the middle of the timeline to the right, labeled "request, 2".
- A straight arrow points from the bottom of the timeline to the right, labeled "release, 4".

At the bottom right, the text "12.3" is displayed.

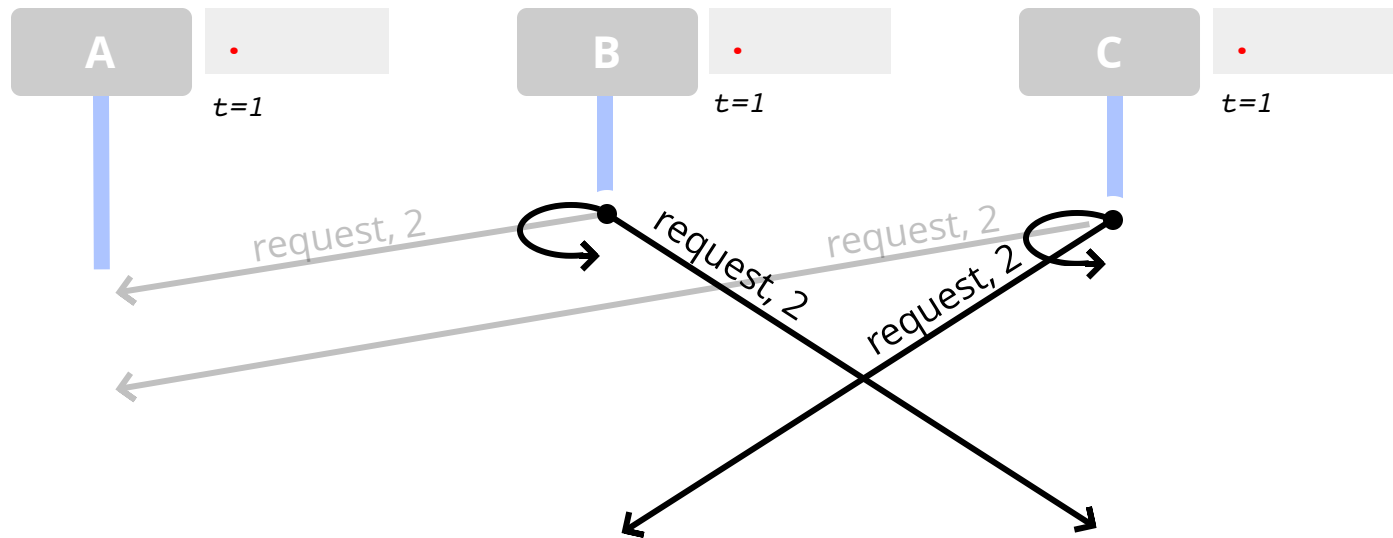
Solution répartie

Version naïve corrigée - Problème



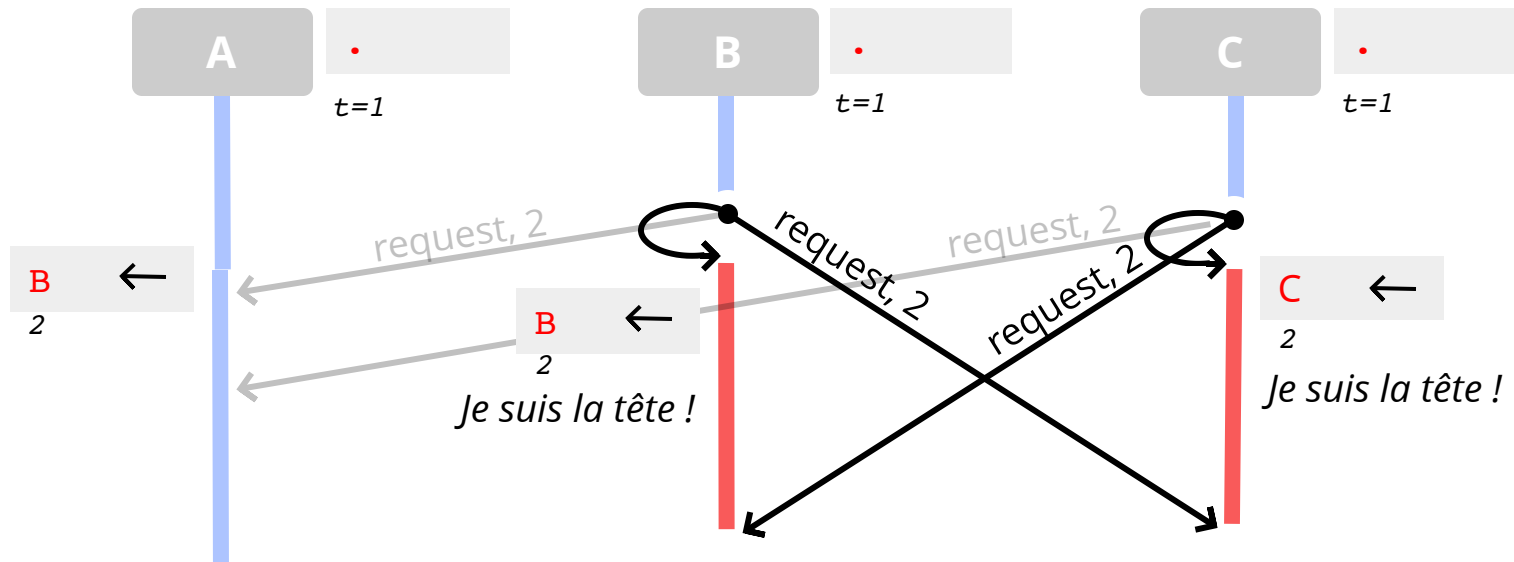
Solution répartie

Version naïve corrigée - Problème



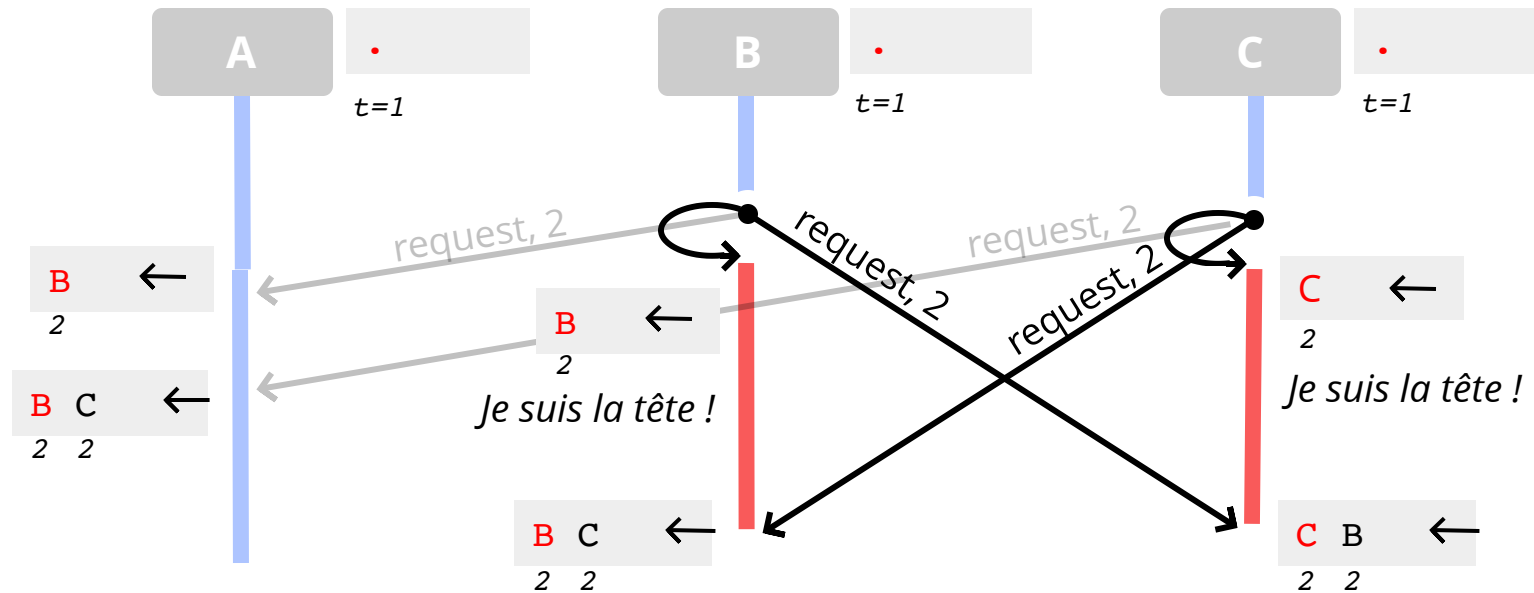
Solution répartie

Version naïve corrigée - Problème



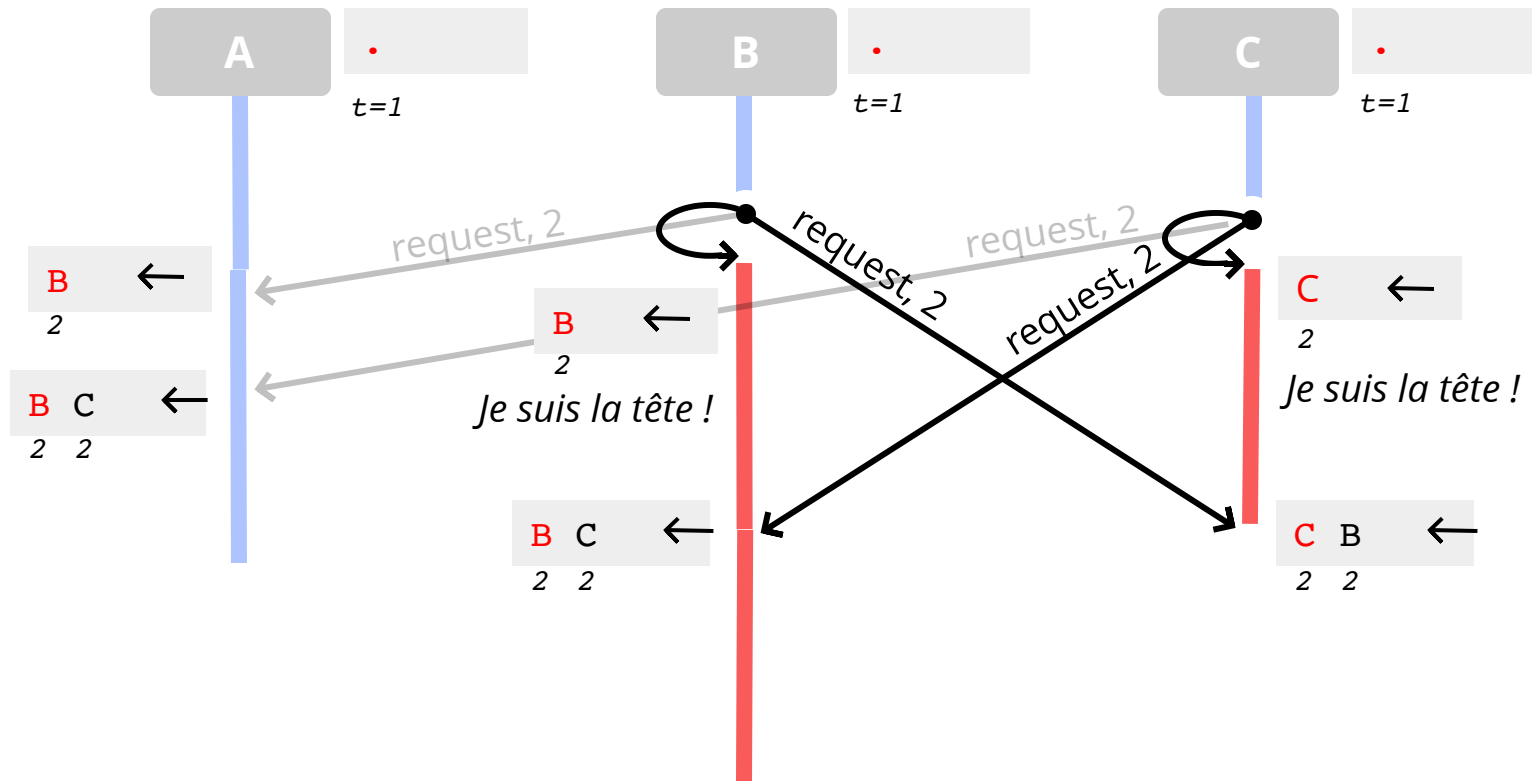
Solution répartie

Version naïve corrigée - Problème



Solution répartie

Version naïve corrigée - Problème

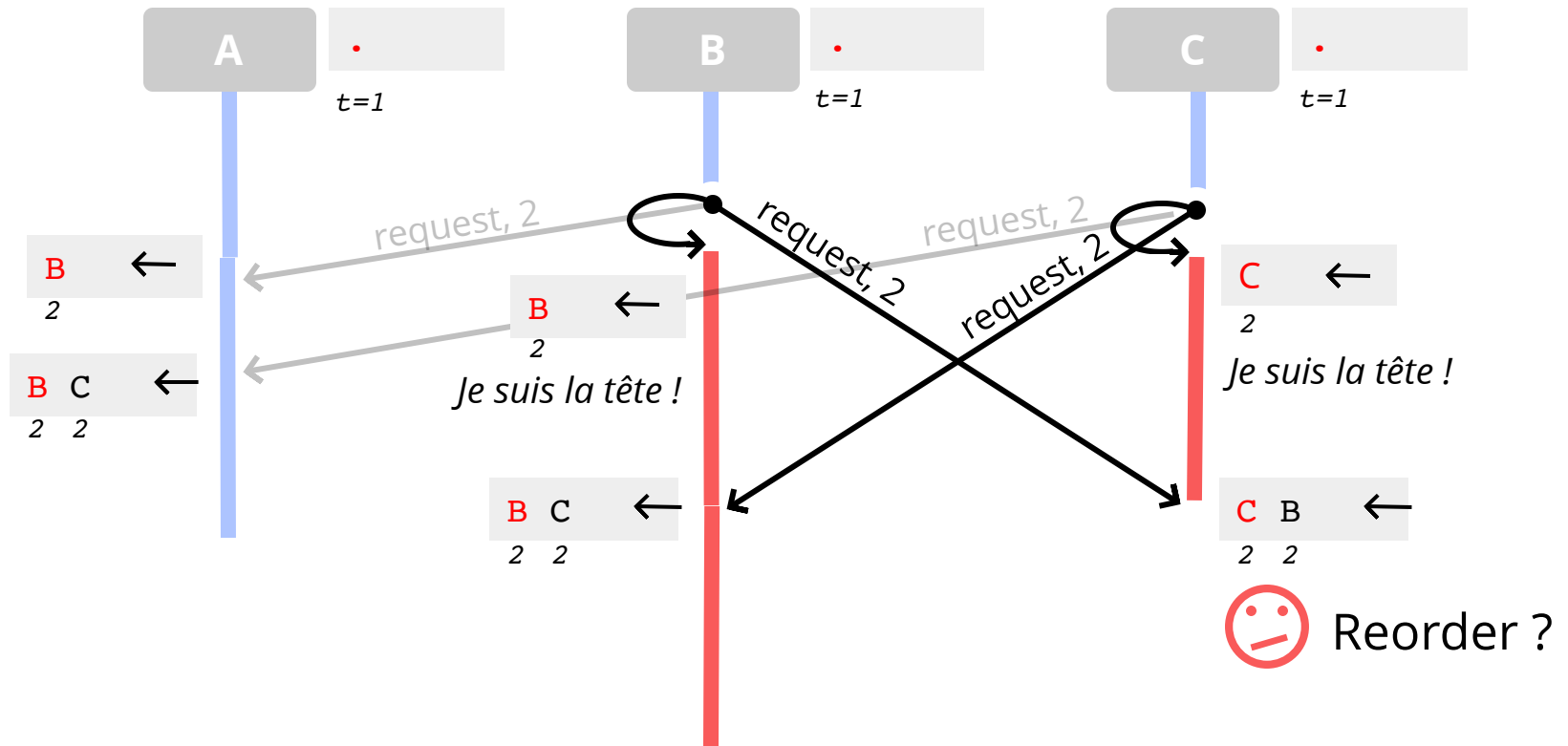


Version naïve corrigée - Problème



Solution répartie

Version naïve corrigée - Problème

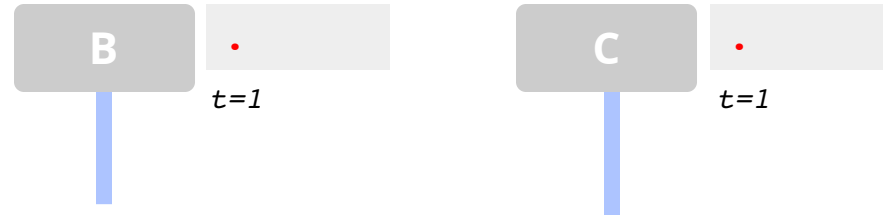


Problème :

Les timestamps de Lamport n'ordonnent pas les messages immédiatement. Ils assurent seulement qu'un ordre strict sera obtenu **un jour**.

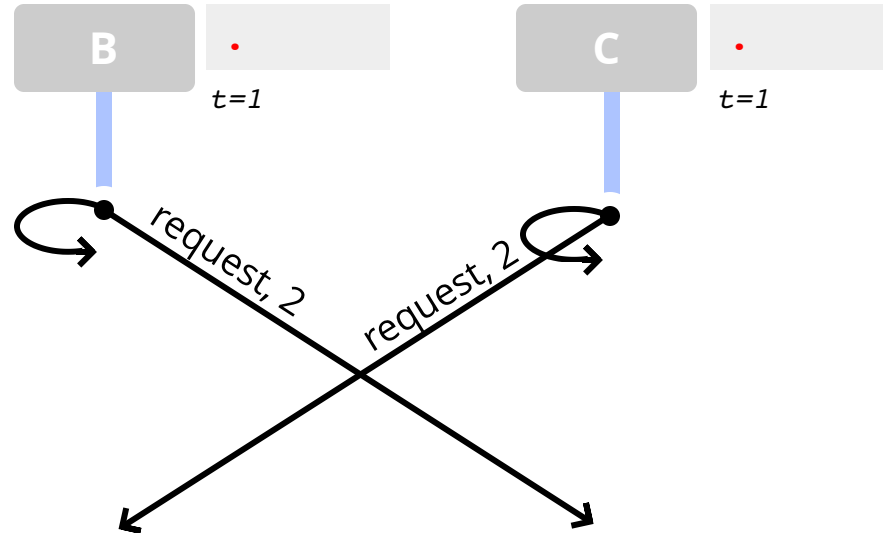
Solution répartie

Version naïve corrigée *corrigée*



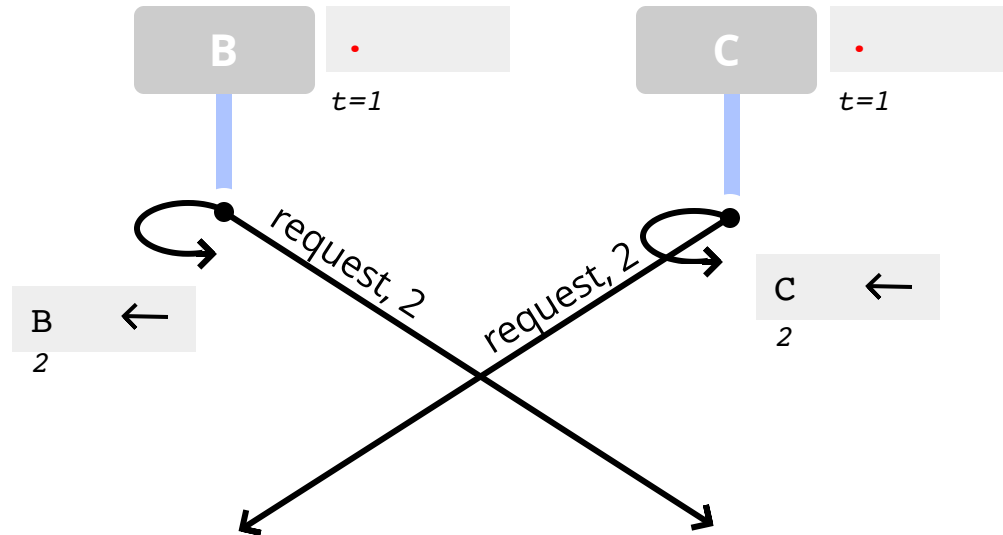
Solution répartie

Version naïve corrigée *corrigée*



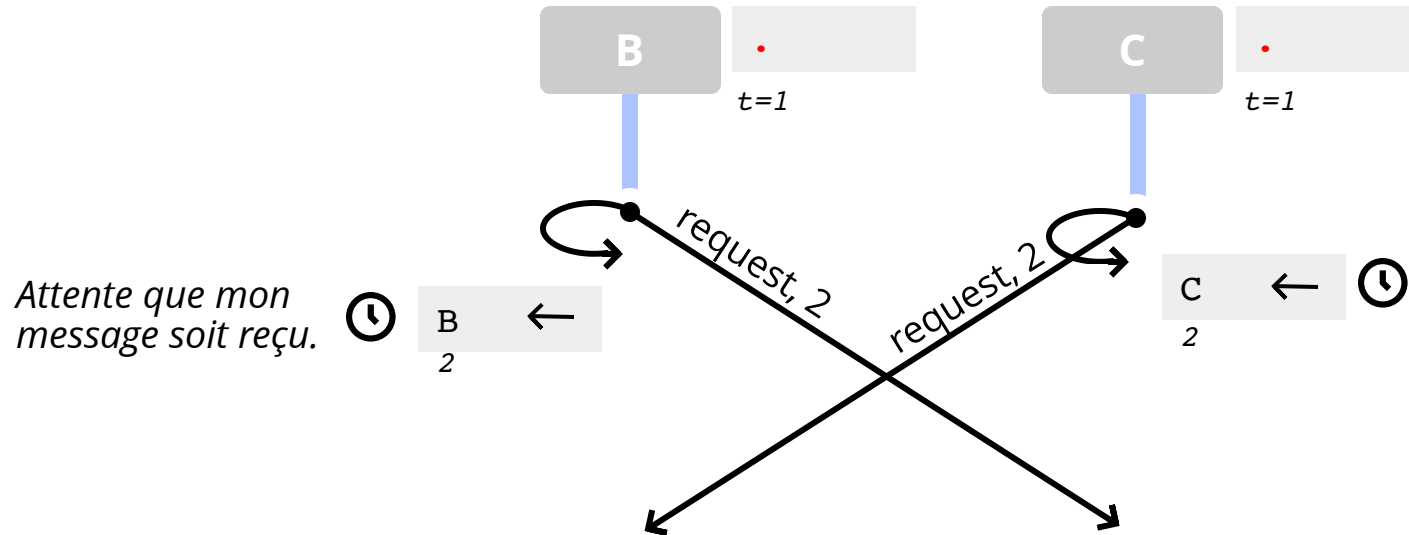
Solution répartie

Version naïve corrigée *corrigée*



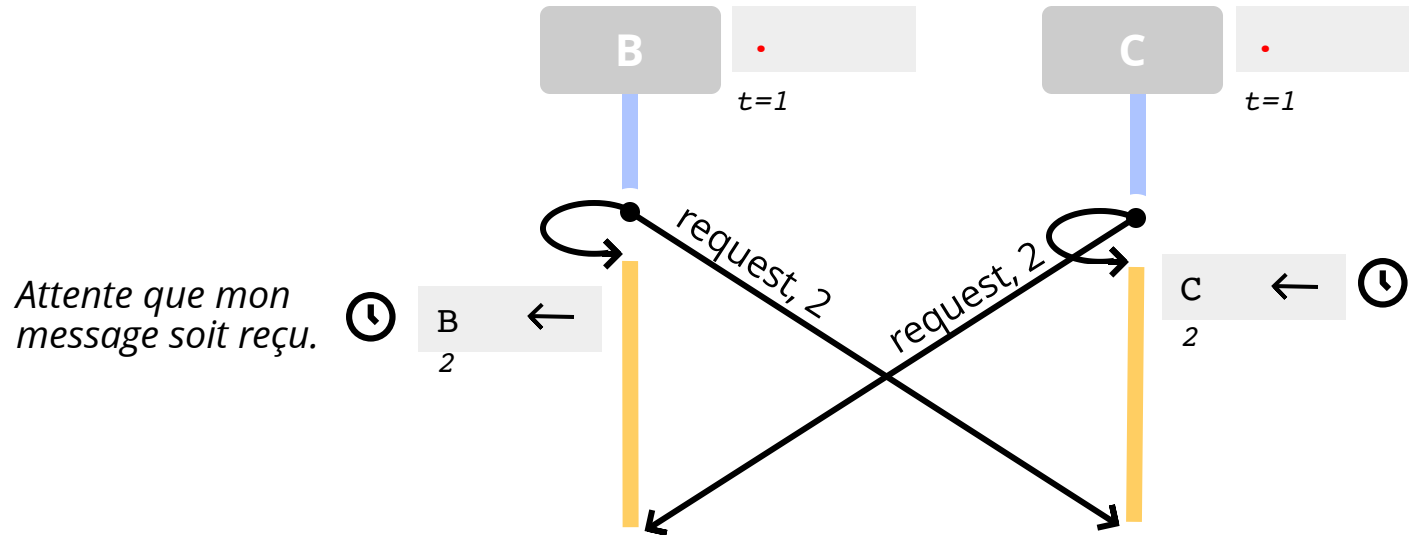
Solution répartie

Version naïve corrigée *corrigée*



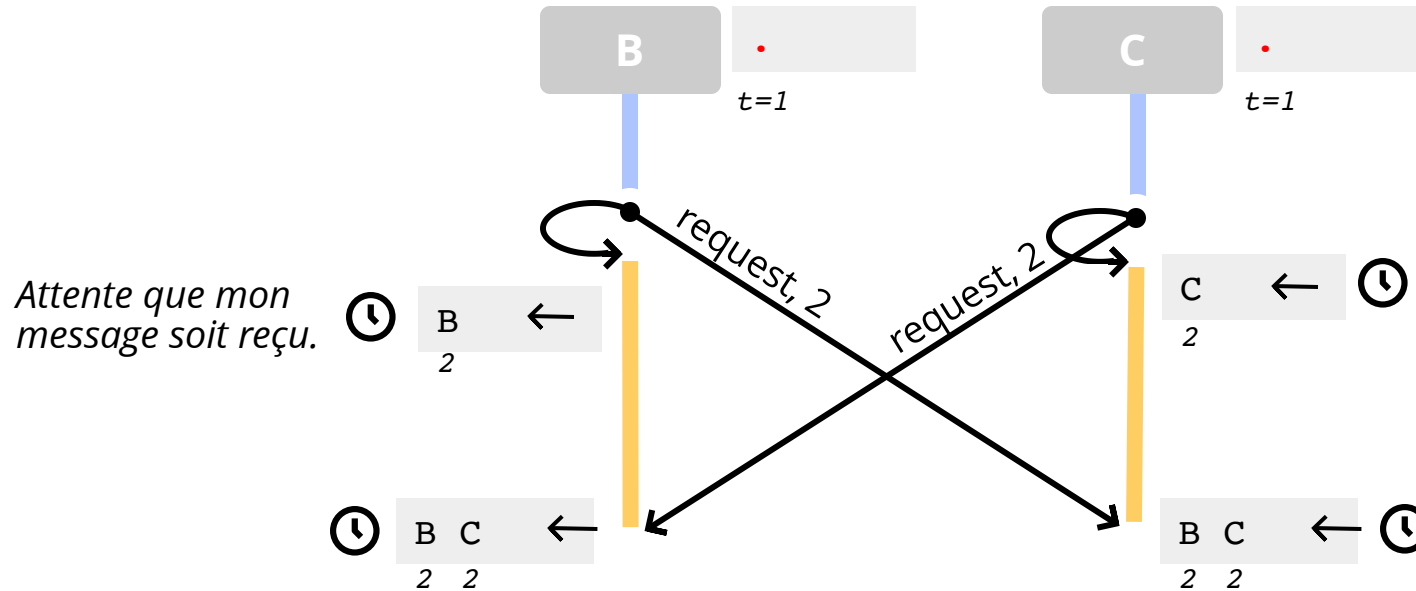
Solution répartie

Version naïve corrigée *corrigée*



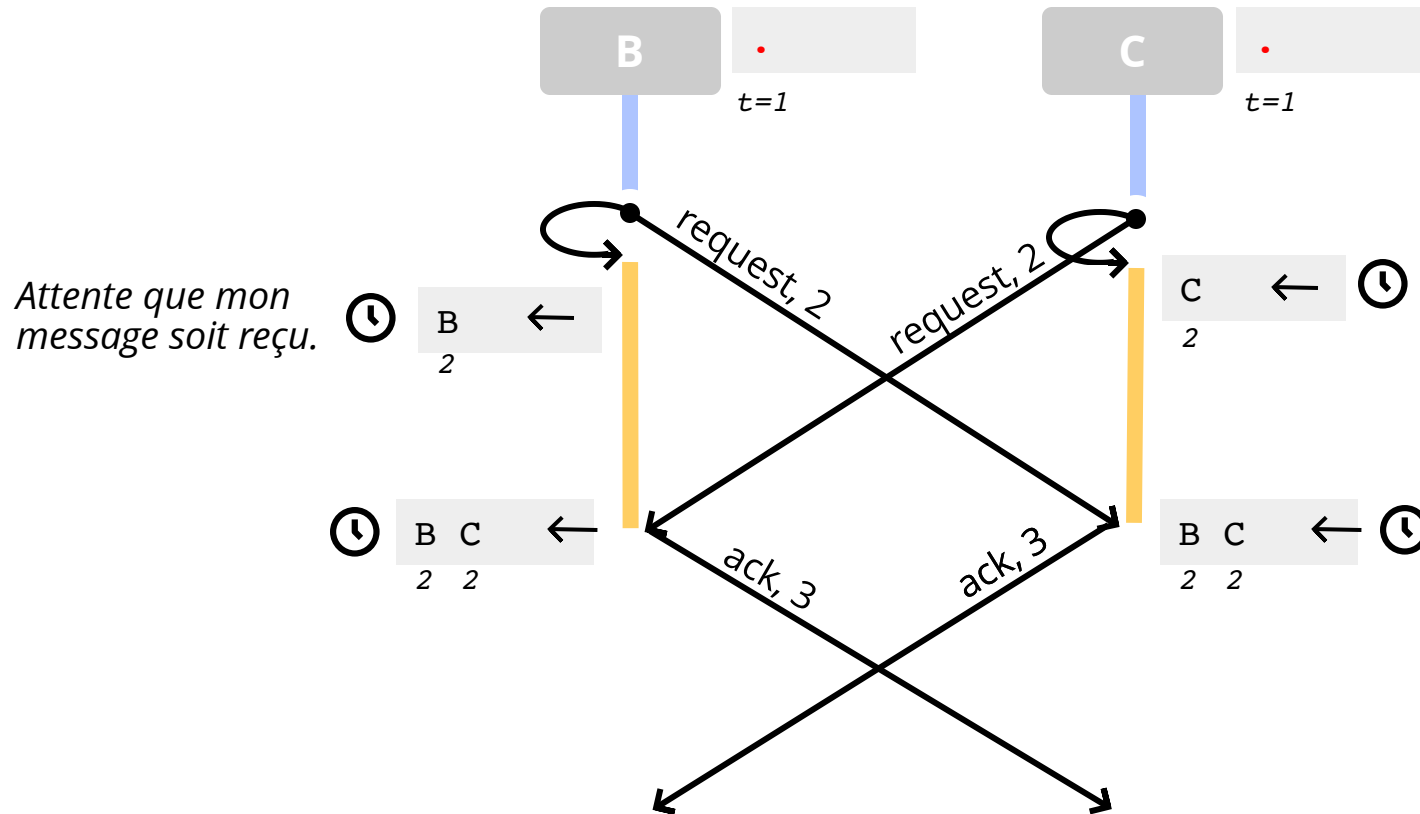
Solution répartie

Version naïve corrigée *corrigée*



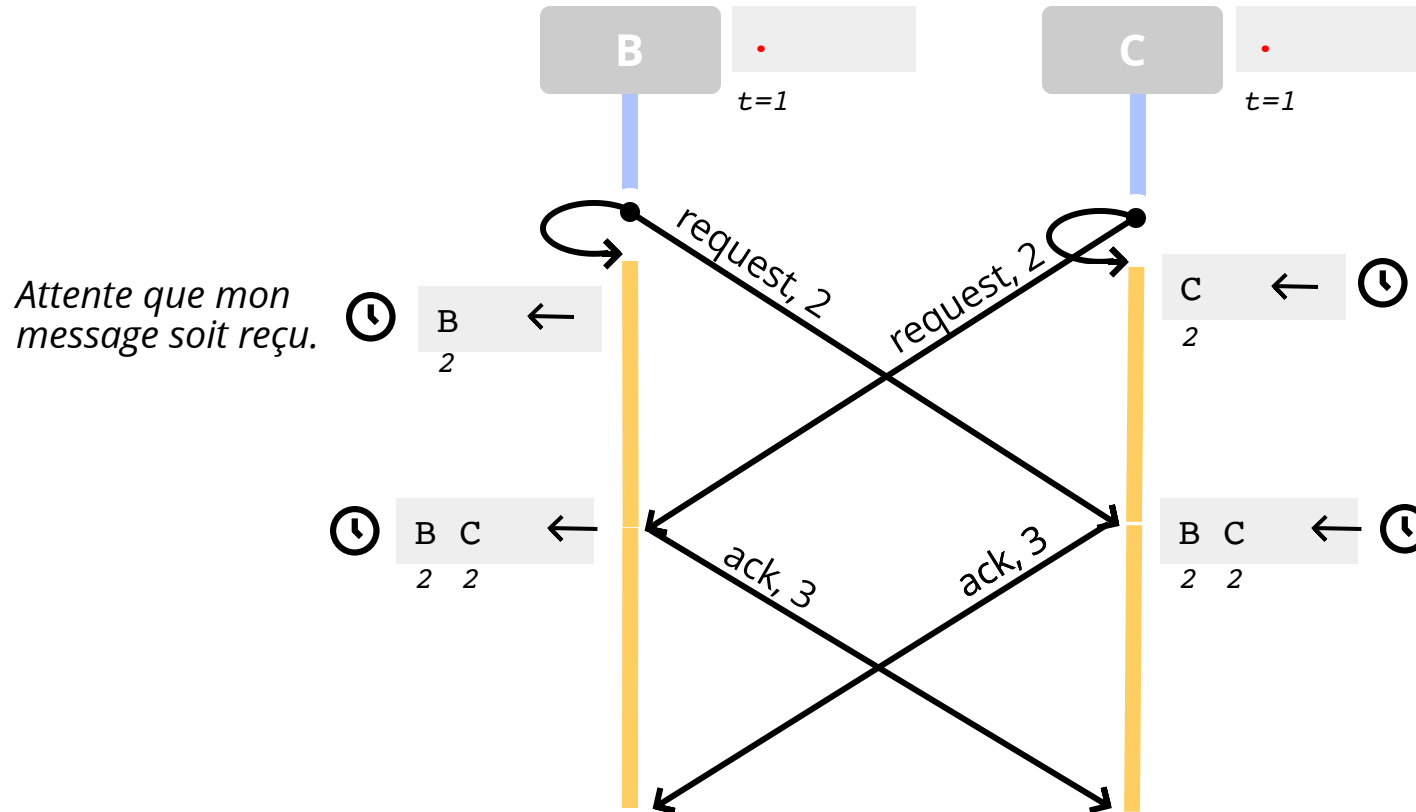
Solution répartie

Version naïve corrigée *corrigée*



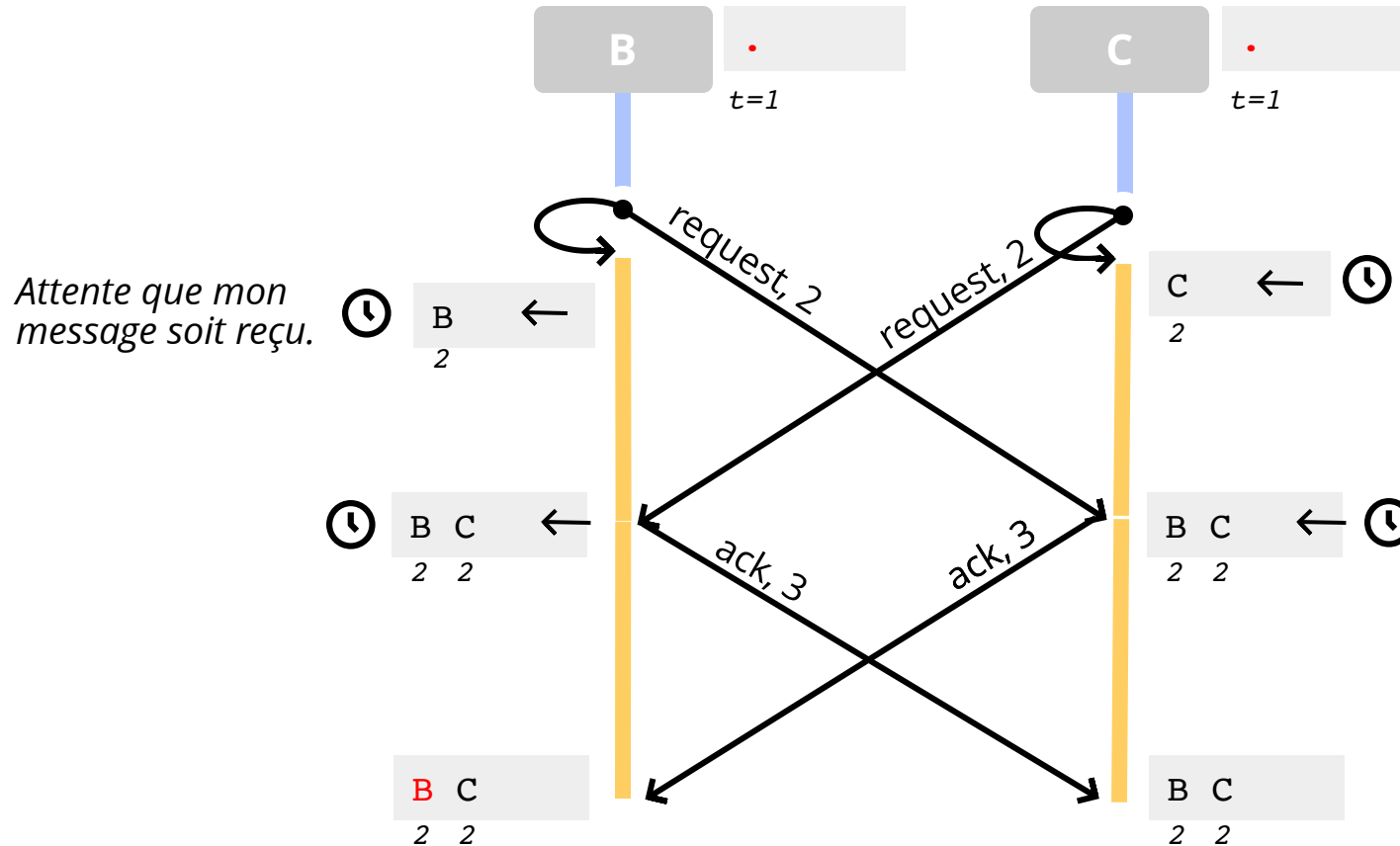
Solution répartie

Version naïve corrigée *corrigée*

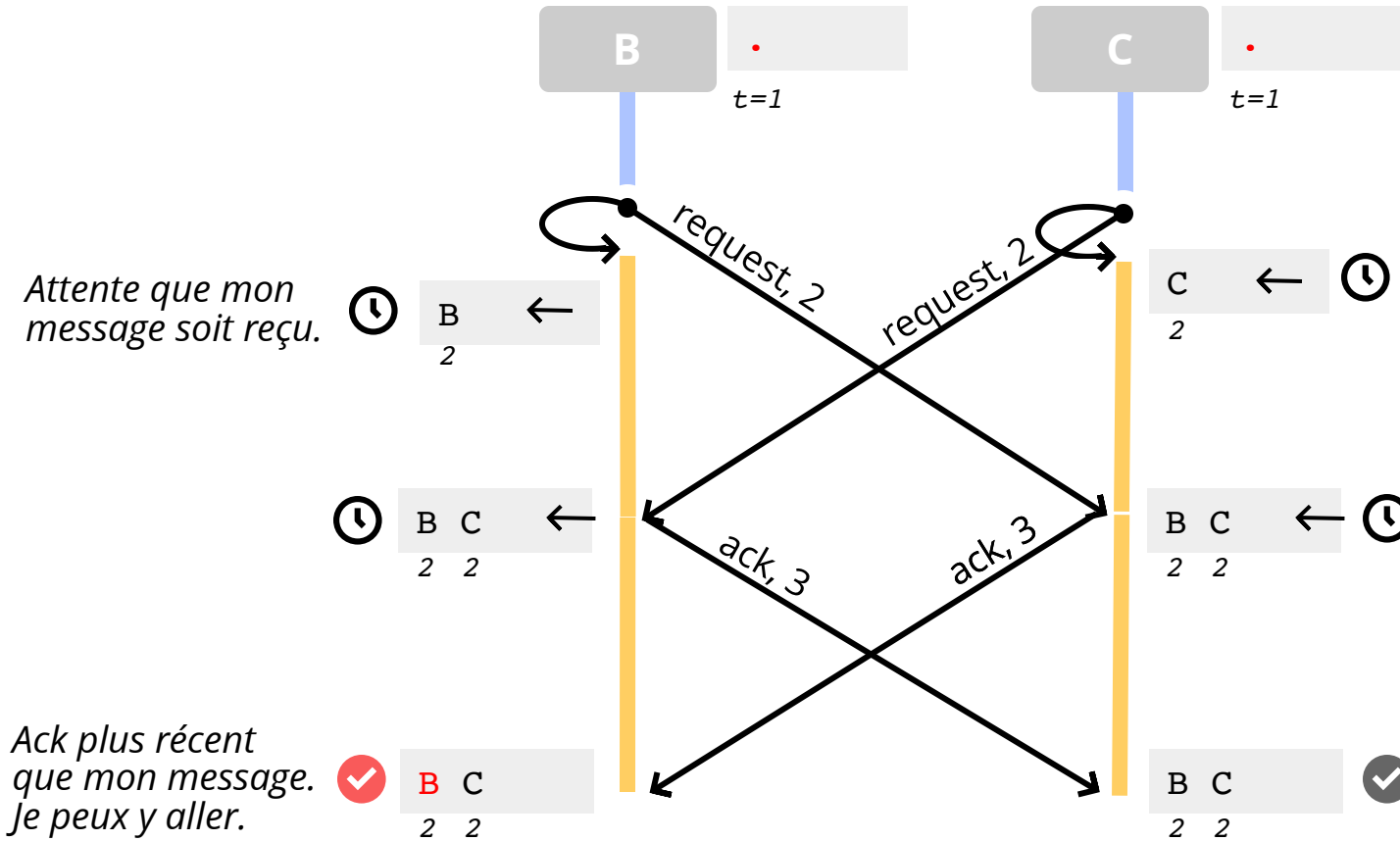


Solution répartie

Version naïve corrigée *corrigée*

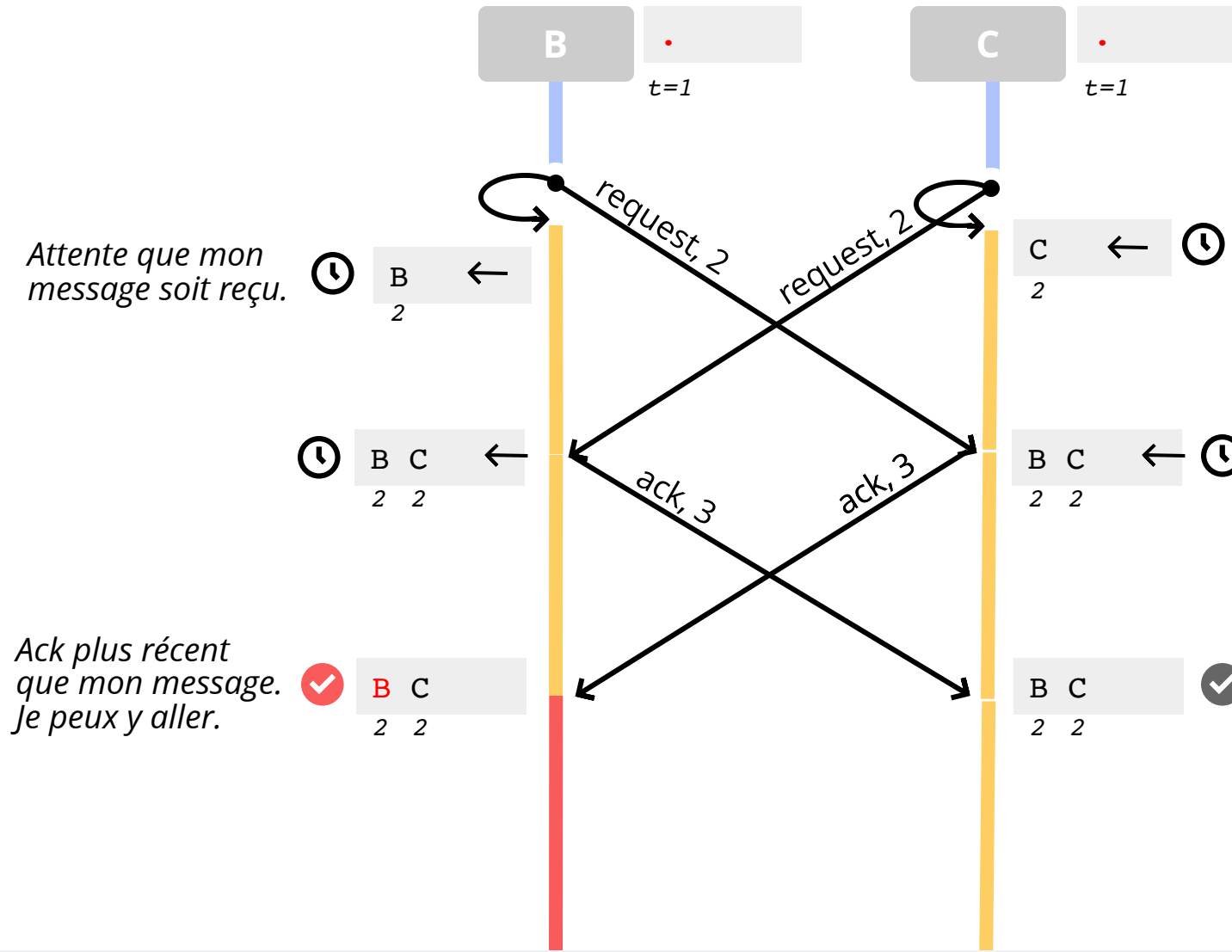


Version naïve corrigée *corrigée*



Solution répartie

Version naïve corrigée *corrigée*



Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

On release

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

On release

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

On release

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file
Entrer en SC

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Entrer en SC

si je suis la nouvelle tête, et
que j'ai reçu tous les acks.

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Entrer en SC
si je suis la nouvelle tête, et
que j'ai reçu tous les acks.

La tête de la file est celle actuellement en SC,

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Entrer en SC
si je suis la nouvelle tête, et
que j'ai reçu tous les acks.

La tête de la file est celle actuellement en SC,
si elle a reçu un ack de toutes les autres machines.

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Entrer en SC
si je suis la nouvelle tête, et
que j'ai reçu tous les acks.

La tête de la file est celle actuellement en SC,
si elle a reçu un ack de toutes les autres machines.

Problème ?

Solution répartie

Version naïve corrigée *corrigée* - Résumé

On request

Push demandeur dans la file
avec son timestamp.

Réordonne la file par heure de
Lamport.

Envoie ack avec nouveau timestamp.

On release

Pop tête de file

Entrer en SC
si je suis la nouvelle tête, et
que j'ai reçu tous les acks.

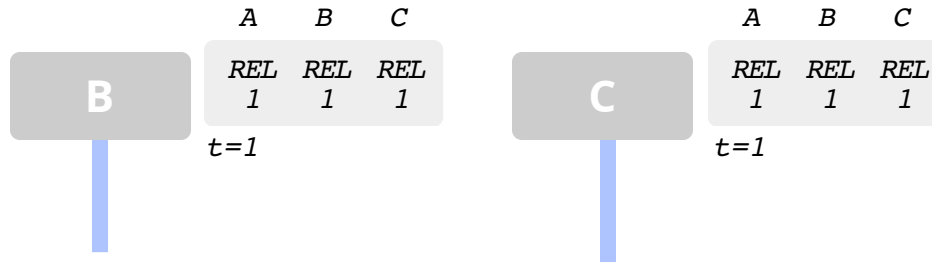
La tête de la file est celle actuellement en SC,
si elle a reçu un ack de toutes les autres machines.

Problème ?

La structure de donnée n'est plus adaptée

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

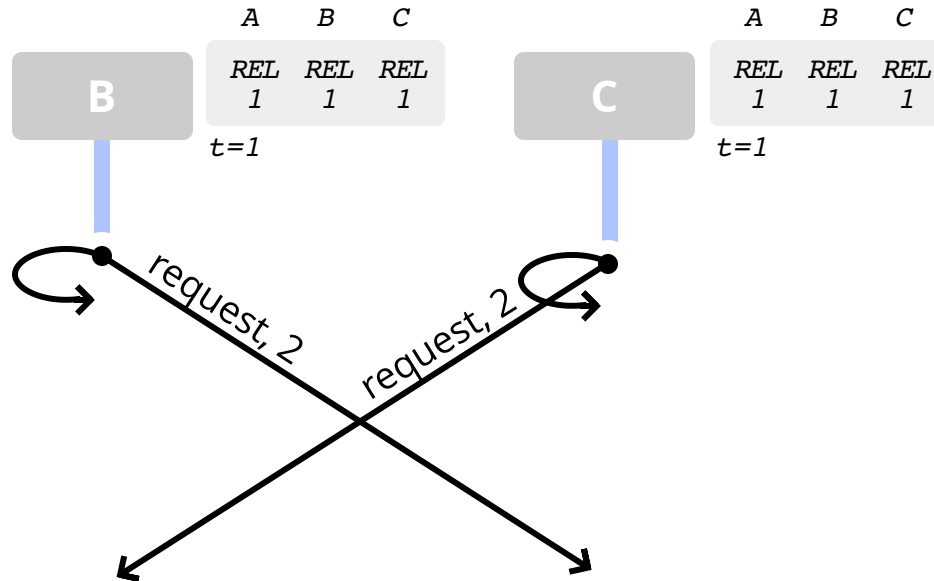
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

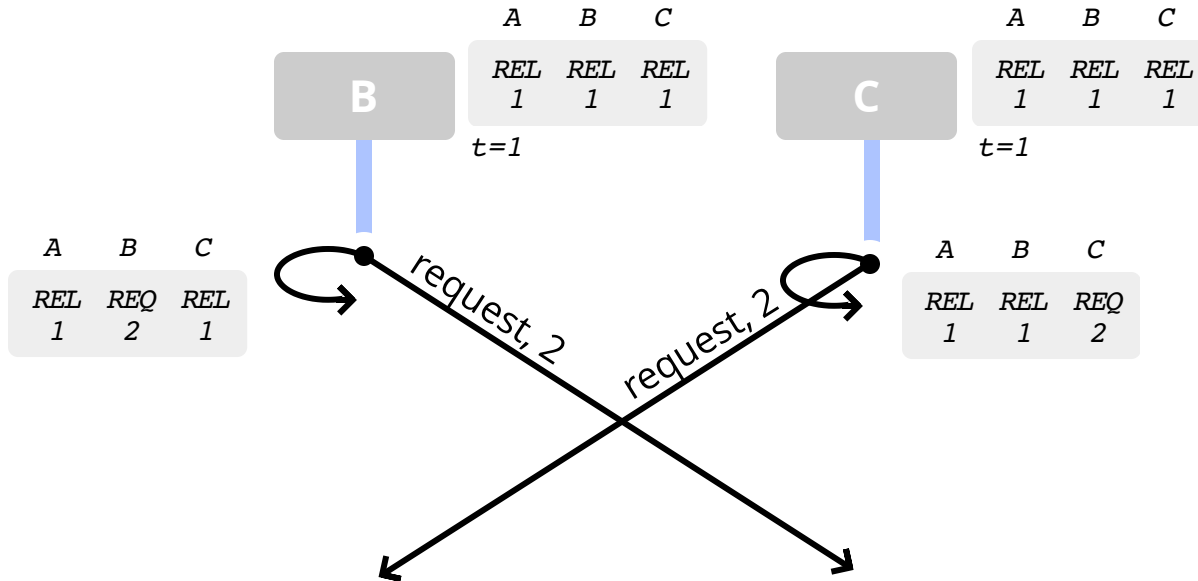
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

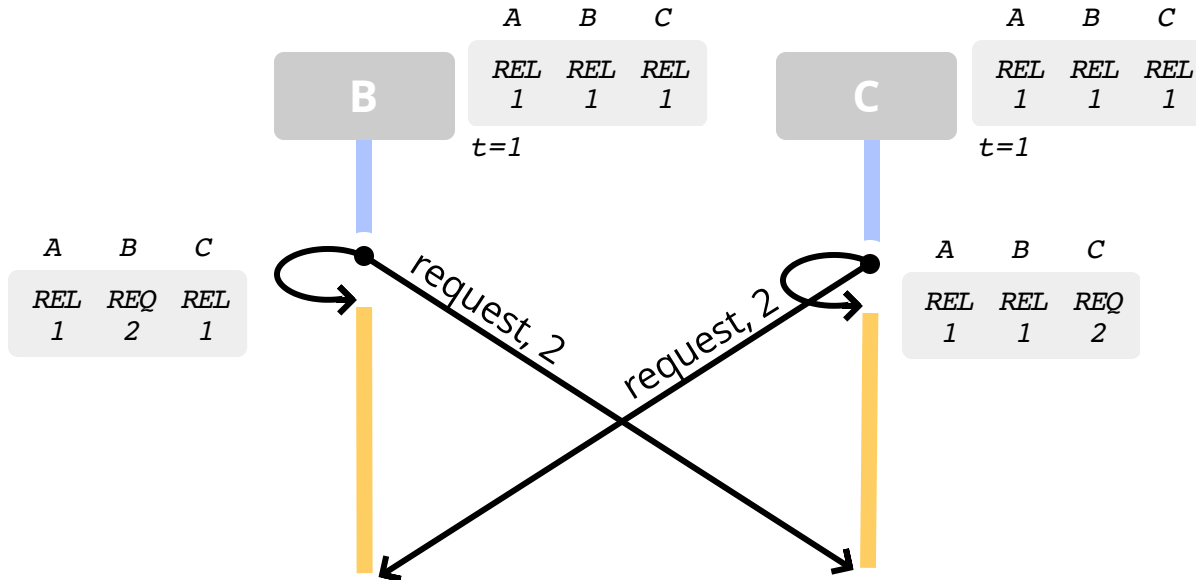
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

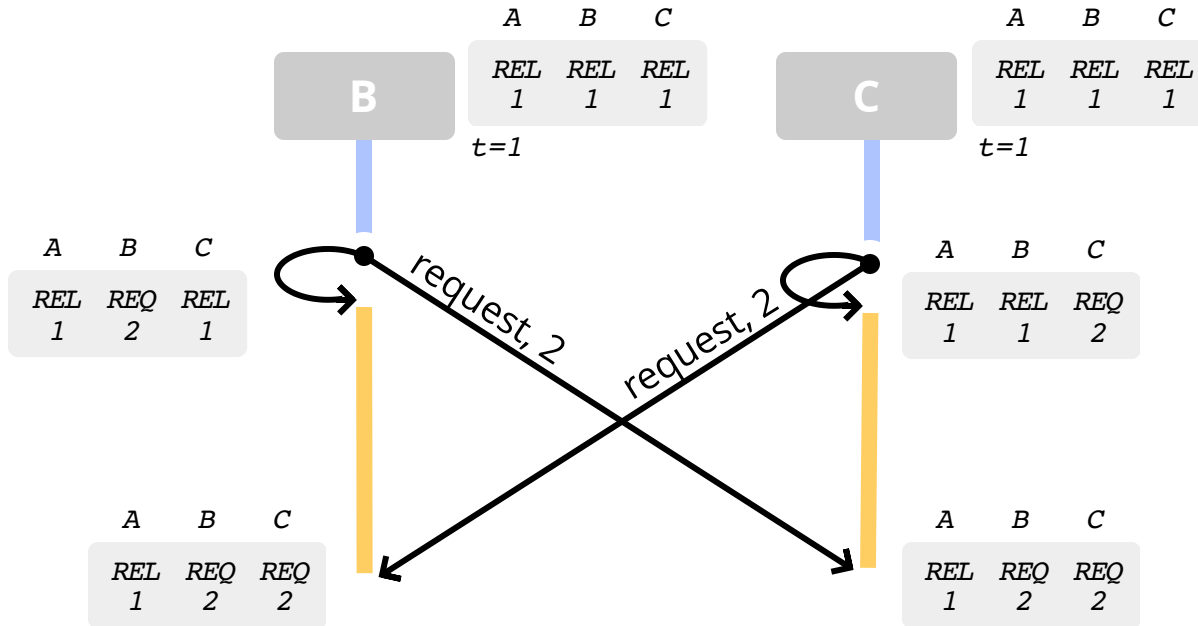
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

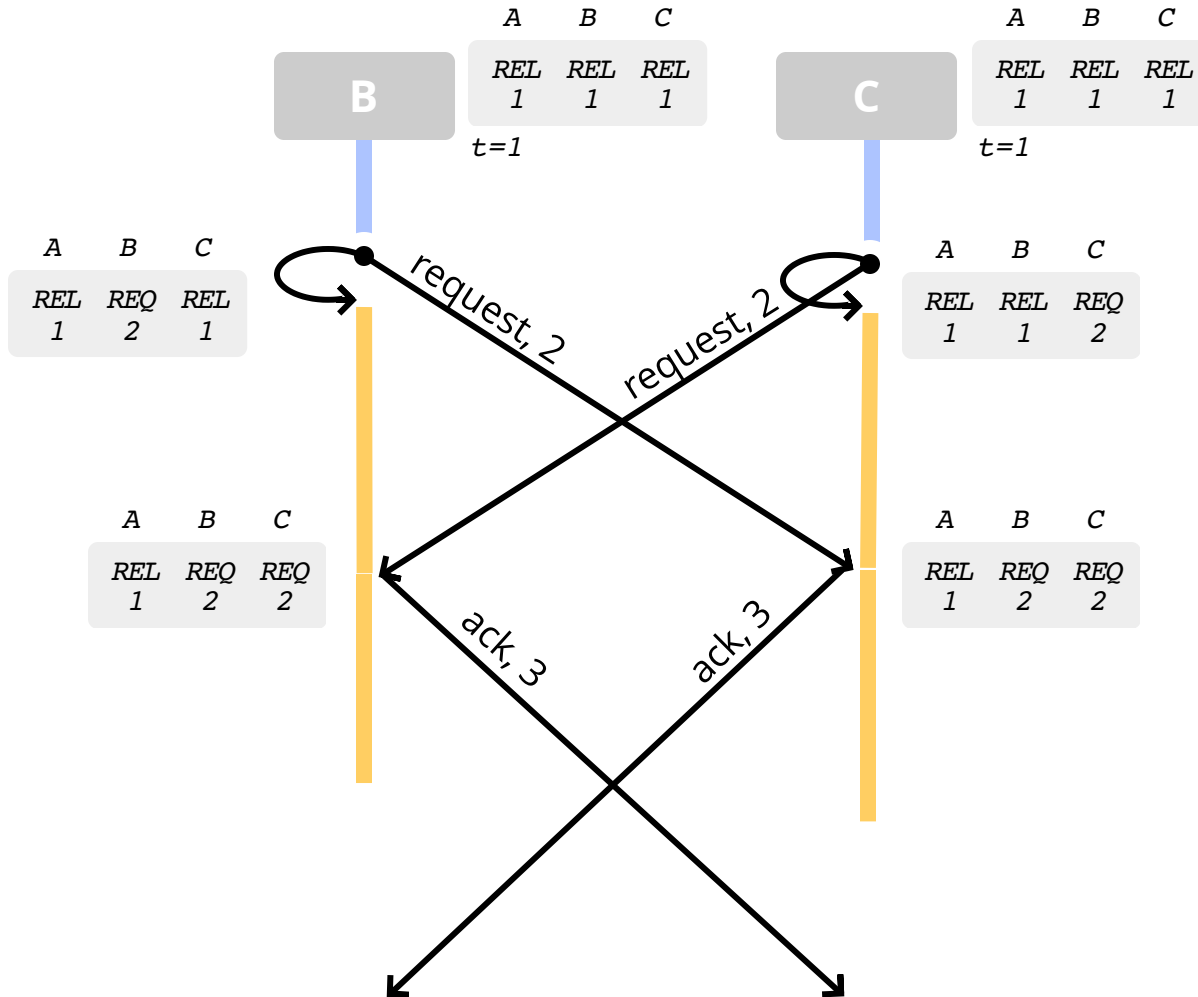
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

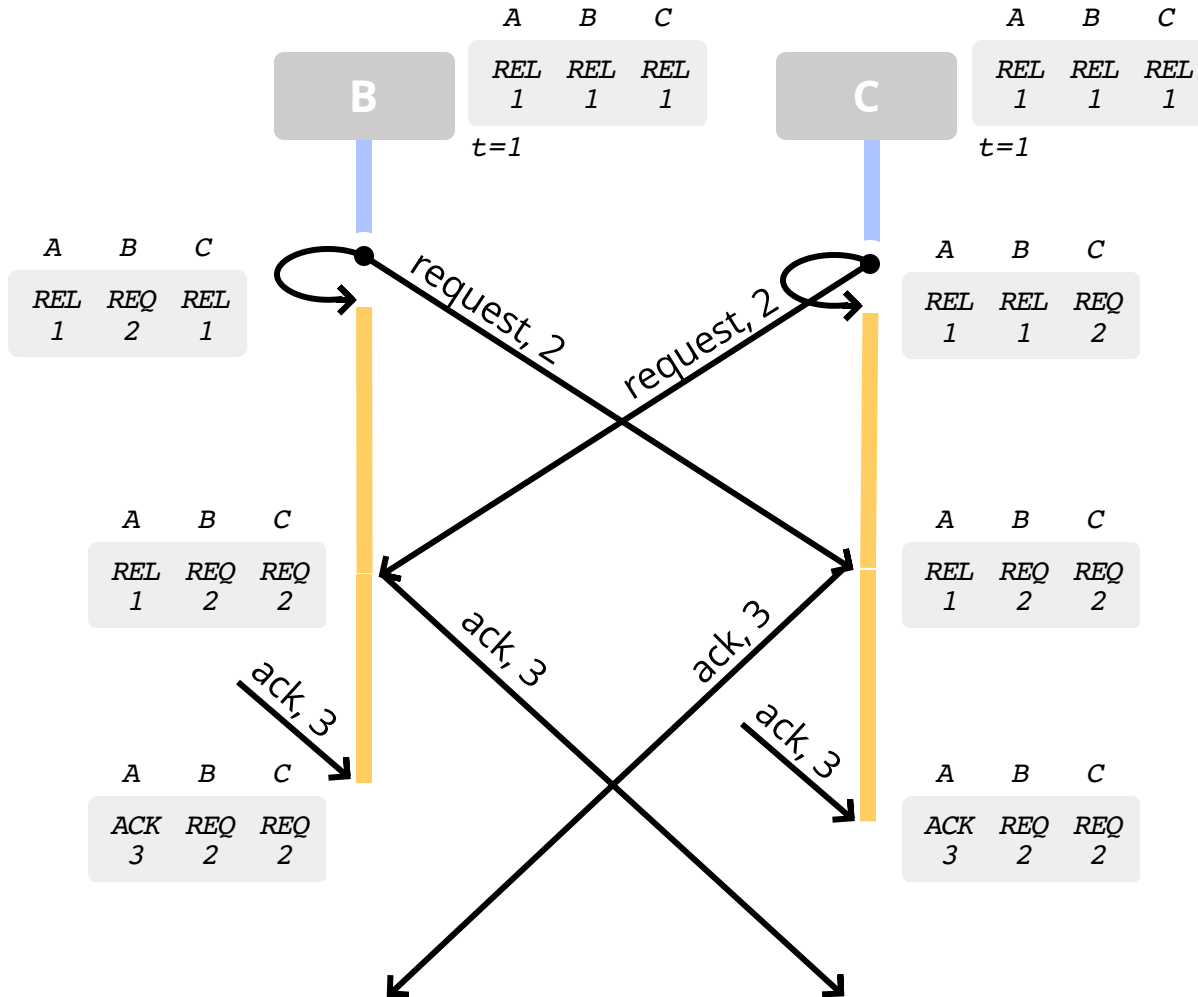
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

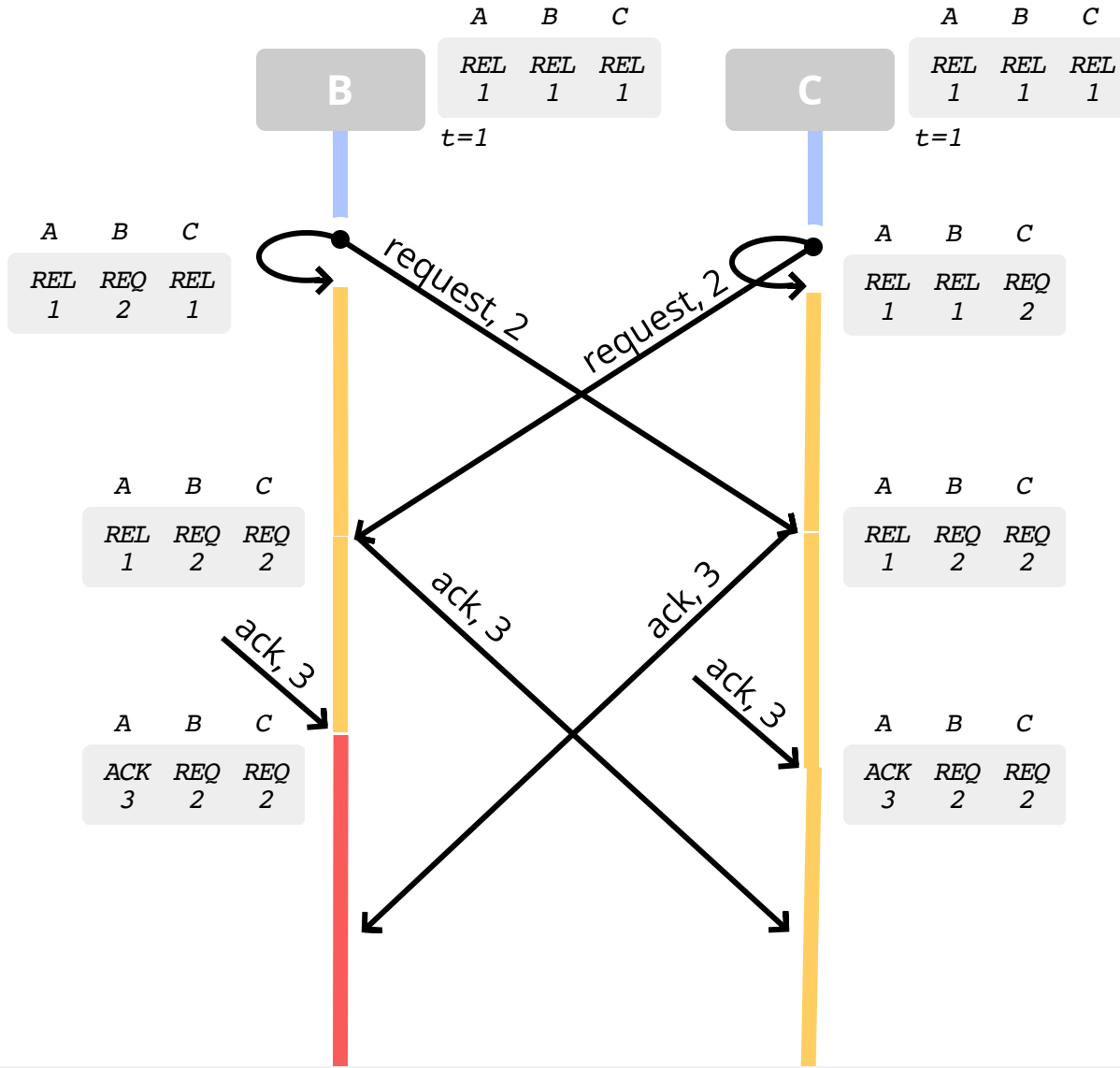
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

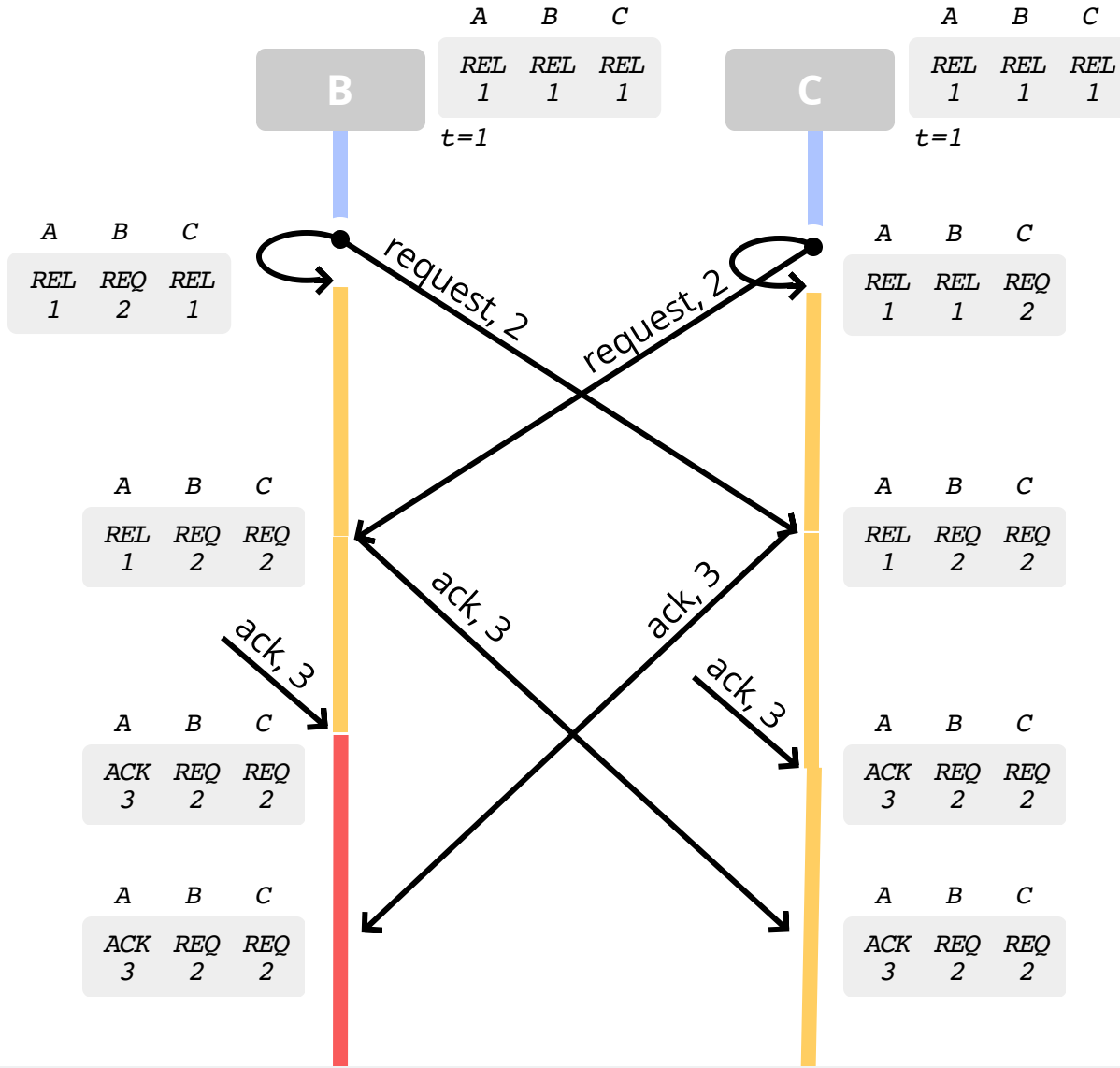
- le dernier message reçu
- le timestamp associé

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?)



Amélioration :

File remplacée par tableau.

Pour chaque processus,
initialement {REL, 1}, puis

- le dernier message reçu
- le timestamp associé

sauf si ACK remplacerait REQ.

Un processus entre en SC si

- son dernier message est REQ,
- et son heure logique est **strictement la plus petite.**

Solution répartie

Version finale (?) - Résumé

On REQ

On REL

On ACK

Après chaque message

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.

On REL

On ACK

Après chaque message

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Après chaque message

On ACK

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

On ACK

Stocke message dans tableau,

Après chaque message

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Entre en SC, si

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Entre en SC, si

- j'ai la plus petite heure logique, et

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Entre en SC, si

- j'ai la plus petite heure logique, et
- c'est un REQ

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Entre en SC, si

- j'ai la plus petite heure logique, et
- c'est un REQ

Suggestions ?

Solution répartie

Version finale (?) - Résumé

On REQ

Stocke message dans tableau.
Envoi ack avec nouveau timestamp.

On REL

Stocke message dans tableau.

On ACK

Stocke message dans tableau,
sauf si un REQ y est déjà.

Après chaque message

Entre en SC, si

- j'ai la plus petite heure logique, et
- c'est un REQ

Suggestions ? *To be continued...*



Algorithme

a.k.a. Algorithme de Lamport

Résumé

Les N processus maintiennent un tableau de taille N , et
un timestamp t_s de Lamport.

Le timestamp est incrémenté en entrée et sortie de SC,
et à toute réception de la part d'un autre processus.

Pour **demander l'entrée en SC**, le processus

- envoie un REQ à tous, lui-même compris, puis attend.

À la **réception de tout message**, le processus

- stocke ce message dans le tableau, dans la case de l'envoyeur,
sauf si cela remplacerait un REQ par un ACK.
- entre en SC si sa case est REQ et a la plus petite heure logique.

À la **réception d'un REQ**, le processus

- envoie un ACK à l'émetteur.

Au **sortir de SC**, le processus

- envoie un REL à tous, lui-même compris.

Pseudocode

Variables

<code>n</code>	<i>entier, constant</i>	nombre de processus
<code>self</code>	<i>entier, entre 0 et n-1</i>	numéro de ce processus
<code>ts</code>	<i>entier, initialement 0</i>	timestamp Lamport de ce processus
<code>enSC</code>	<i>booléen</i>	vrai ssi autorisé à être en SC
<code>tab</code>	<i>tableau de messages</i>	messages de chaque processus de format $\{ [REQ ACK REL], ts \}$, initialement $\{REL, 0\}$.

Pseudocode

Initialisation

Écouter infiniment les événements suivants :

- Demande de SC de la couche applicative

- Passage de `enSC` à `true`

- Sortie de SC dans la couche applicative

- Réception `REQ(ts)`

- Réception `ACK(ts)`

- Réception `REL(ts)`

(Avec traitement séquentiel de chaque événement)

Pseudocode

Traitement : Demande de SC de la couche applicative.

`ts` incrémenté.

Envoi de `{REQ, ts}` à tous les processus, `self` inclus.

Traitement : Passage de `enSC` à `true`.

Autorisation de la couche applicative à entrer en SC.

Traitement : Sortie de SC dans la couche applicative.

`ts` incrémenté.

Envoi de `{REL, ts}` à tous les processus, `self` inclus.

Pseudocode

Traitement : Réception {REQ, ts_i } du processus i .

$ts \leftarrow \max(ts_i, ts) + 1$, si i n'est pas self.

$tab[i] \leftarrow \{REQ, ts_i\}$.

Envoi de {ACK, ts } à i .

Tentative d'entrée en SC.

Traitement : Réception {REL, ts_i } du processus i .

$ts \leftarrow \max(ts_i, ts) + 1$, si i n'est pas self.

$tab[i] \leftarrow \{REL, ts_i\}$.

Tentative d'entrée en SC.

Traitement : Réception {ACK, ts_i } du processus i .

$ts \leftarrow \max(ts_i, ts) + 1$, si i n'est pas self.

Si $tab[i]$ n'a pas comme type REQ,

$tab[i] \leftarrow \{ACK, ts_i\}$.

Tentative d'entrée en SC.

Pseudocode

Tentative d'entrée en SC.

```
Si tab[self] est de type REQ  
  Extraire toutes les heures logiques dans tab.  
  Si la plus ancienne appartient à self,  
    enSC  $\leftarrow$  true
```

↓ **Propriétés**

Propriétés

Algorithme de Lamport

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

Complexité

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

Complexité

Communication par processus souhaitant entrer en SC :

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

Complexité

Communication par processus souhaitant entrer en SC : $3(n-1)$

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

Complexité

Communication par processus souhaitant entrer en SC : $3(n-1)$

Calcul par événement :

Propriétés

Algorithme de Lamport

Correctness Jamais plus d'un processus ne sera en SC à un instant T.

Progrès Toute demande d'entrée en SC sera autorisée un jour.

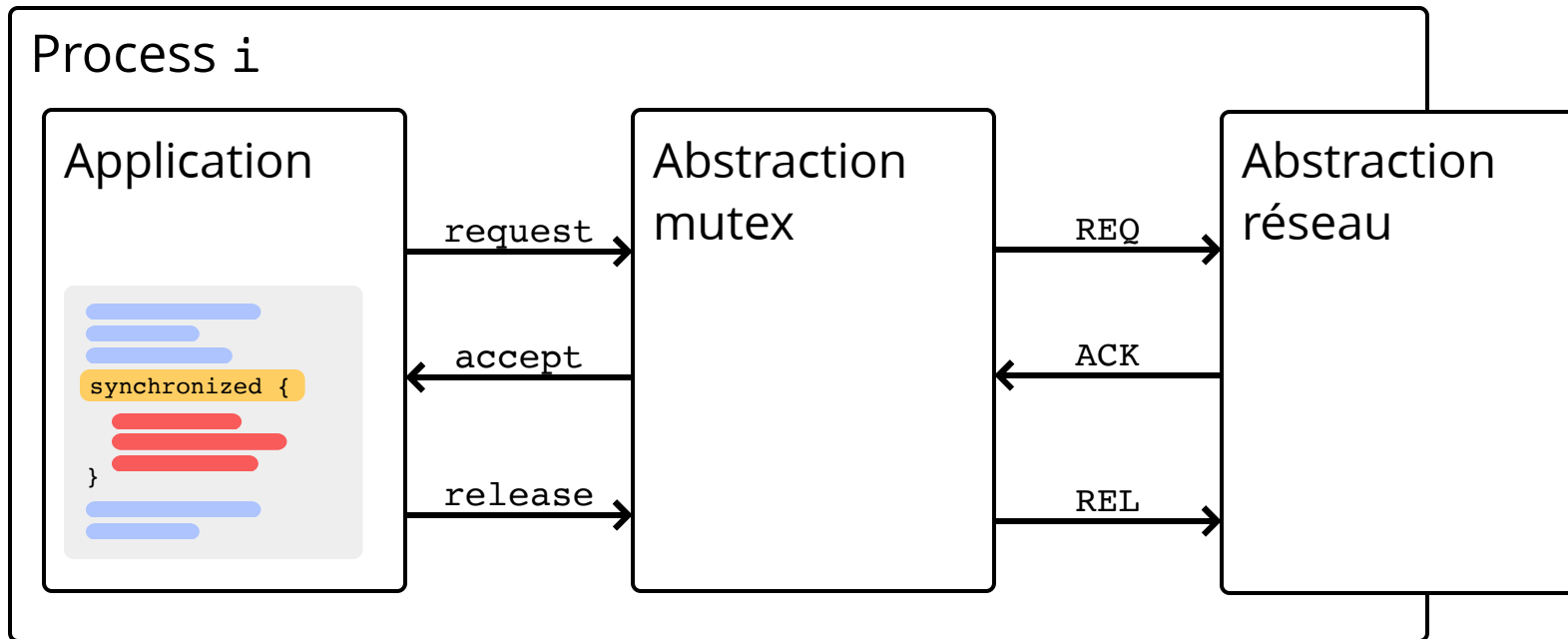
Complexité

Communication par processus souhaitant entrer en SC : $3(n-1)$

Calcul par événement : $O(n)$ par réception de message
 $O(1)$ pour le reste.

↓ **Implémentation logicielle**

Implémentation logicielle



Implémenté comme une couche d'abstraction de mutex.

Couche d'abstraction réseau : indépendance de l'abstraction mutex et du système de communication choisi (tcp, udp, channels, etc).

↓ Exercices

Exercice 1

Suppositions:

- Un délai de transit de 2s existe pour toute communication.
- Tout autre délai est relativement négligeable (e.g. traitement, copie, etc.)
- Une SC dure 5s.
- Les événements simultanés sont priorisés dans l'ordre suivant :
 - demande de SC, sortie de SC, gestion de message
 - le nom de l'émetteur sert à trier deux messages reçus simultanément
- Deux processus dans le système, et initialement
 - A a un timestamp à 2,
 - B a un timestamp à 10.

A demandera la SC au bout d'une seconde, et B au bout de 4s.

Exercice 2

Mêmes conditions, mais

Trois processus, et initialement

- A a un timestamp à 10,
- B a un timestamp à 6,
- C a un timestamp à 4.

Demandes de SC

- A au bout de 1 seconde,
- C au bout de 2 secondes.

Exercice 3

Amélioration ?

Ça fait beaucoup de messages...

Y a-t-il des messages qui peuvent être omis ?

Modifiez le pseudocode en conséquence.